

COMPUTER SCIENCE GRADES 1 TO 12

Curriculum Guide Elementary, Junior High & Senior High

Original teaching resource NOT an official Ministry of Education syllabus transcription; no official Liberian Computer Science syllabus was supplied.

INTRODUCTION

Computer Science equips every learner to give, follow, check and improve instructions with people, on paper and on machines. The course is unplugged-first: a school without a computer is not a school without Computer Science. Elementary grades (1–6) build computing ideas through cards, charts and paper keyboards; junior high (7–9) is paper-first with trace tables and paired debugging; senior high (10–12) extends the same habits into formal computer science. Each grade carries six four-week units, Periods I to VI, three per semester.

The general objectives for Grades 1–12 Computer Science:

1. Use computing vocabulary accurately in speech and in writing.
2. Follow, write and debug algorithms with a partner, unplugged or on paper.
3. Apply computational thinking patterns, steps, counts and conversions to unfamiliar problems.
4. Use devices and information safely, fairly and lawfully.
5. Work as driver and navigator, swapping roles, and debug without blame.
6. Connect computing ideas with Liberian daily life, work and careers.

A learner-centred, unplugged-first approach is emphasised: learners take ownership through pair programming, debugging together, classroom scenarios and short written algorithms. Safeguarding runs through every unit – pupils never share passwords, photographs or home addresses; a report of online bullying is listened to once, written in the child's words and taken to the head teacher the same day.

Source status: no official Liberian Computer Science syllabus was supplied for this repository, so these units are original teaching resources aligned to the shape of a West African computer-science course – not a transcription of an official curriculum or examination. Digital research links are omitted by design so the course works fully offline.

SEMESTER ONE**GRADE: 1 PERIOD: I****TOPIC: Computers Around Us**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Computers Around Us and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	A computer is a machine that follows instructions. In Liberia a computer may be a laptop in a school office, a phone in a market stall, or the radio that plays the news. A class with no computer can still compute: we give clear instructions to one another and to paper machines. Key content: 1. computer a machine that follows instructions to work with information. 2. instruction a clear step that tells a person or a machine what to do next. 3. device a tool that uses electricity to compute, store or send information. 4. screen the part of a device that shows pictures, words and other output. 5. phone a handheld device used to call, message and run programs. 6. laptop a portable computer with a screen and a keyboard together. 7. radio a way of sending sound without wires, still widely used in Liberia. 8. market a place where people buy and sell, often using phones for prices and payments. 9. user a person who gives instructions to a computer. 10. rule an agreed instruction that keeps people and devices safe. 11. input information or an action that goes into a computer. 12. output information or a result that comes out of a computer. 13. unplugged computing practised with paper, cards, bodies or talk, not a machine. 14. algorithm a finite set of steps that solves a problem or finishes a task.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Human computers in a market line . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		15. hardware the physical parts of a computer that you can touch. 16. model a simpler version of a real thing used to think or to practise. 17. step one action in a sequence of instructions. 18. safety protection from foreseeable harm, including online harm.				

SEMESTER ONE**GRADE: 1 PERIOD: II****TOPIC: Following Instructions**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Following Instructions and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	An algorithm is a finite set of steps that finishes a task. A recipe for palava sauce, the way we line up for assembly, and the way we open a book are all algorithms. If a step is missing or in the wrong order, the result is wrong. Key content: 1. instruction a clear step that tells a person or a machine what to do next. 2. algorithm a finite set of steps that solves a problem or finishes a task. 3. sequence steps that must be done in a fixed order. 4. rule an agreed instruction that keeps people and devices safe. 5. turn the chance to use a shared device or to give the next instruction. 6. pair two pupils working together on one set of instructions. 7. user a person who gives instructions to a computer. 8. computer a machine that follows instructions to work with information. 9. step one action in a sequence of instructions. 10. thinking working out a problem carefully before acting. 11. test trying a set of instructions with known inputs to see if they work.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of From the door to the well . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER ONE**GRADE: 1 PERIOD: III****TOPIC: Patterns We Repeat**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Patterns We Repeat and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	A pattern is something that repeats in a regular way. Clapping 1-2-3, 1-2-3 is a loop. Computers are good at repeating a step without getting tired, which is why a loop is one of the first ideas in computing. Key content: 1. pattern something that repeats in a regular, predictable way. 2. sequence steps that must be done in a fixed order. 3. loop a part of an algorithm that repeats a step a number of times. 4. iteration repeating a step until a job is finished. 5. rule an agreed instruction that keeps people and devices safe. 6. algorithm a finite set of steps that solves a problem or finishes a task. 7. instruction a clear step that tells a person or a machine what to do next. 8. computer a machine that follows instructions to work with information. 9. clap a repeated sound used as a beat or a counted loop. 10. turn the chance to use a shared device or to give the next instruction. 11. step one action in a sequence of instructions.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Clap, step, repeat . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER TWO**GRADE: 1 PERIOD: IV****TOPIC: Parts of a Computer**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Parts of a Computer and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	Hardware is the physical parts of a computer that you can touch. A keyboard, a mouse, a screen, a printer and the box that holds the processor are hardware. Even a paper keyboard on a desk teaches the same names. Key content: 1. hardware the physical parts of a computer that you can touch. 2. keyboard a board of keys used to type letters, numbers and commands. 3. mouse a pointing device used to move a pointer and choose things on a screen. 4. screen the part of a device that shows pictures, words and other output. 5. processor the part of a computer that follows instructions. 6. memory the part of a computer that holds information while work is happening. 7. storage the place where a computer keeps files for later use. 8. laptop a portable computer with a screen and a keyboard together. 9. desktop a computer that stays on a desk and is not meant to be carried. 10. printer a machine that puts computer output onto paper.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Paper computer on the desk . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER TWO**GRADE: 1 PERIOD: V****TOPIC: Being Safe with Machines**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Being Safe with Machines and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	Machines need dry hands, a safe place to sit, and agreed rules. Food and drink stay away from keyboards. A charger cable is not a toy. If a device feels hot, smells of burning, or a stranger asks for a picture, stop and tell the teacher. Key content: 1. safety protection from foreseeable harm, including online harm. 2. rule an agreed instruction that keeps people and devices safe. 3. device a tool that uses electricity to compute, store or send information. 4. cable a wire that carries power or information between devices. 5. screen the part of a device that shows pictures, words and other output. 6. phone a handheld device used to call, message and run programs. 7. laptop a portable computer with a screen and a keyboard together. 8. user a person who gives instructions to a computer. 9. turn the chance to use a shared device or to give the next instruction. 10. report to tell a trusted adult about harm, a threat or a worrying message. 11. charger the cable and plug that give a device electrical power.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of The classroom device rules . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER TWO**GRADE: 1 PERIOD: VI****TOPIC: Taking Turns with Devices**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Taking Turns with Devices and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	A school may have one laptop or none. Taking turns is a computing skill: one person drives, one person navigates, and they swap. Nobody grabs, nobody laughs at a slow typist, and the next pair leaves the work saved. Key content: 1. turn the chance to use a shared device or to give the next instruction. 2. pair two pupils working together on one set of instructions. 3. collaboration working with others on one computing task. 4. rule an agreed instruction that keeps people and devices safe. 5. device a tool that uses electricity to compute, store or send information. 6. share to let another person see or use information. 7. user a person who gives instructions to a computer. 8. computer a machine that follows instructions to work with information. 9. keyboard a board of keys used to type letters, numbers and commands. 10. mouse a pointing device used to move a pointer and choose things on a screen. 11. save to store work so it can be opened again later.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Driver and navigator . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER ONE**GRADE: 2 PERIOD: 1****TOPIC: Input and Output**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Input and Output and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	Input is what goes in; output is what comes out. Pressing a key is input; letters on the screen are output. A camera takes a picture in; a printer puts a page out. Naming input and output is the start of understanding any machine. Key content: 1. input information or an action that goes into a computer. 2. output information or a result that comes out of a computer. 3. keyboard a board of keys used to type letters, numbers and commands. 4. mouse a pointing device used to move a pointer and choose things on a screen. 5. screen the part of a device that shows pictures, words and other output. 6. speaker a device that turns computer output into sound. 7. printer a machine that puts computer output onto paper. 8. camera a device that captures a picture as digital information. 9. sensor a part that notices light, sound, heat or movement and reports it. 10. device a tool that uses electricity to compute, store or send information.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of What goes in, what comes out . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER ONE**GRADE: 2 PERIOD: II****TOPIC: Step-by-Step Sequences**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Step-by-Step Sequences and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	A sequence is steps in a fixed order. Brushing teeth, making rice, and starting a laptop are sequences. If you pour water before you put the pot on the fire, the result changes. Flowcharts show sequence with boxes and arrows. Key content: 1. sequence steps that must be done in a fixed order. 2. algorithm a finite set of steps that solves a problem or finishes a task. 3. instruction a clear step that tells a person or a machine what to do next. 4. flowchart a diagram of boxes and arrows that shows the order of steps. 5. step one action in a sequence of instructions. 6. thinking working out a problem carefully before acting. 7. program a set of instructions a computer can follow. 8. user a person who gives instructions to a computer. 9. test trying a set of instructions with known inputs to see if they work. 10. debug to find and fix a mistake in a set of instructions. 11. bug a mistake in instructions that makes the result wrong.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Make rice in order . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER ONE**GRADE: 2 PERIOD: III****TOPIC: Information versus Things**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Information versus Things and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	A mango is a thing. The word 'mango', a tally of mangoes sold, and a drawing of a mango are information. Information can be copied without using up the original. Things cannot. Computing works with information. Key content: 1. information data that has been organised so it can answer a question. 2. data recorded facts or numbers that can be organised and used. 3. computer a machine that follows instructions to work with information. 4. file a named collection of information stored as one item. 5. message information sent from one person or device to another. 6. copy to make a second version of information without removing the first. 7. save to store work so it can be opened again later. 8. storage the place where a computer keeps files for later use. 9. bit the smallest unit of digital information, either 0 or 1. 10. byte a group of eight bits, enough to store one ordinary character. 11. privacy keeping personal information away from people who should not see it. 12. consent freely given agreement before something is done with a person's information.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Market tally versus market goods . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER TWO**GRADE: 2 PERIOD: IV****TOPIC: The Keyboard and the Mouse**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in The Keyboard and the Mouse and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	A keyboard puts letters into a computer. A mouse moves a pointer so we can click, drag and choose an icon. On a paper keyboard we still find A, space and enter. Slow, correct typing beats fast, wrong typing. Key content: 1. keyboard a board of keys used to type letters, numbers and commands. 2. mouse a pointing device used to move a pointer and choose things on a screen. 3. click pressing a mouse button or a screen to choose something. 4. type pressing keys to put letters and numbers into a computer. 5. drag moving an item on a screen by holding and sliding it. 6. icon a small picture that stands for a program, file or action. 7. window a rectangular area on a screen that shows one program or file. 8. menu a list of choices a program offers. 9. screen the part of a device that shows pictures, words and other output. 10. input information or an action that goes into a computer. 11. hardware the physical parts of a computer that you can touch.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Paper keyboard treasure hunt . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER TWO**GRADE: 2 PERIOD: V****TOPIC: Being Kind with Technology**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Being Kind with Technology and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	A message can travel farther than a voice. Kind computing means we do not send a joke that hurts, we do not share a classmate's picture, and we report bullying to a trusted adult. A screen does not hide unkindness. Key content: 1. rule an agreed instruction that keeps people and devices safe. 2. safety protection from foreseeable harm, including online harm. 3. share to let another person see or use information. 4. message information sent from one person or device to another. 5. bully a person who repeatedly harms or frightens someone, including through a device. 6. report to tell a trusted adult about harm, a threat or a worrying message. 7. privacy keeping personal information away from people who should not see it. 8. consent freely given agreement before something is done with a person's information. 9. user a person who gives instructions to a computer. 10. citizen a person who uses computing tools with care for other people. 11. digital citizen a person who uses computing tools with care for other people.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Kind words, kind clicks . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER TWO**GRADE: 2 PERIOD: VI****TOPIC: Saving Our Work**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in 'Saving Our Work' and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	Work that is not saved can disappear when a device sleeps or the power goes. A file needs a name a person can read later, such as 'Mary-story-Monday'. A folder holds related files. Saving is a habit, not a last step. Key content: 1. save to store work so it can be opened again later. 2. file a named collection of information stored as one item. 3. folder a named place that holds files and other folders. 4. storage the place where a computer keeps files for later use. 5. copy to make a second version of information without removing the first. 6. name the word we give a file, a folder or a variable so we can find it again. 7. user a person who gives instructions to a computer. 8. account a named space that holds a person's files, settings and history. 9. login the steps that prove who you are before a device opens your work. 10. computer a machine that follows instructions to work with information. 11. memory the part of a computer that holds information while work is happening.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Name it, then save it . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER ONE**GRADE: 3 PERIOD: I****TOPIC: Repeating Steps (Loops)**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Repeating Steps (Loops) and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	A loop repeats a step until a job is finished or a count is reached. 'Clap four times' is a counted loop. 'Keep sweeping until the floor is clean' is a loop that stops when a test is true. Loops save us from writing the same step again and again. Key content: 1. loop a part of an algorithm that repeats a step a number of times. 2. iteration repeating a step until a job is finished. 3. sequence steps that must be done in a fixed order. 4. algorithm a finite set of steps that solves a problem or finishes a task. 5. pattern something that repeats in a regular, predictable way. 6. instruction a clear step that tells a person or a machine what to do next. 7. program a set of instructions a computer can follow. 8. flowchart a diagram of boxes and arrows that shows the order of steps. 9. test trying a set of instructions with known inputs to see if they work. 10. thinking working out a problem carefully before acting. 11. bug a mistake in instructions that makes the result wrong. 12. debug to find and fix a mistake in a set of instructions.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Cross the twelve-square grid . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER ONE**GRADE: 3 PERIOD: II****TOPIC: Finding and Fixing Mistakes**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Finding and Fixing Mistakes and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	A bug is a mistake in the instructions. Debugging is finding and fixing it without blaming the person who wrote it. We try, we debug, we do not laugh at a mistake. Read the steps aloud, act them out, and change one thing at a time. Key content: 1. bug a mistake in instructions that makes the result wrong. 2. debug to find and fix a mistake in a set of instructions. 3. test trying a set of instructions with known inputs to see if they work. 4. algorithm a finite set of steps that solves a problem or finishes a task. 5. sequence steps that must be done in a fixed order. 6. instruction a clear step that tells a person or a machine what to do next. 7. pair two pupils working together on one set of instructions. 8. thinking working out a problem carefully before acting. 9. flowchart a diagram of boxes and arrows that shows the order of steps. 10. program a set of instructions a computer can follow.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Debug the paper robot . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER ONE**GRADE: 3 PERIOD: III****TOPIC: Files and Folders**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Files and Folders and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	A file is one named item. A folder is a named place that holds files. A messy desktop is like a desk with every paper in one heap. Sorting files into folders named by subject and week makes searching faster. Key content: 1. file a named collection of information stored as one item. 2. folder a named place that holds files and other folders. 3. storage the place where a computer keeps files for later use. 4. save to store work so it can be opened again later. 5. name the word we give a file, a folder or a variable so we can find it again. 6. copy to make a second version of information without removing the first. 7. desktop a computer that stays on a desk and is not meant to be carried. 8. window a rectangular area on a screen that shows one program or file. 9. icon a small picture that stands for a program, file or action. 10. sort to put items into an agreed order. 11. search looking for information by using key words.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Sort the class desktop . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER TWO**GRADE: 3 PERIOD: IV****TOPIC: The Internet as a Library**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in The Internet as a Library and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	The internet is a worldwide network that lets computers share information. It is a library that never closes and a library that also holds mistakes and unkind pages. A website is a collection of pages. A browser is the program that shows them. Check who wrote a page before you trust it. Key content: 1. internet a worldwide network that lets computers share information. 2. website a collection of pages on the internet that belong together. 3. browser a program used to look at pages on the internet. 4. link a word or picture that opens another page or file when chosen. 5. search looking for information by using key words. 6. source the person, page or record from which information comes. 7. evidence information that can support or challenge a claim. 8. verification checking a claim against a reliable source. 9. information data that has been organised so it can answer a question. 10. wifi a wireless way for a device to join a network.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Who wrote this page? . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER TWO**GRADE: 3 PERIOD: V****TOPIC: Coding Without a Computer**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Coding Without a Computer and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	Unplugged computing uses paper, cards, grids and our bodies. A paper robot, a sorting-hat of cards, and a human flowchart are real code. A school without a computer is not a school without Computer Science. Key content: 1. unplugged computing practised with paper, cards, bodies or talk, not a machine. 2. code instructions written in a language a computer can follow. 3. algorithm a finite set of steps that solves a problem or finishes a task. 4. flowchart a diagram of boxes and arrows that shows the order of steps. 5. program a set of instructions a computer can follow. 6. instruction a clear step that tells a person or a machine what to do next. 7. sequence steps that must be done in a fixed order. 8. pair two pupils working together on one set of instructions. 9. model a simpler version of a real thing used to think or to practise. 10. thinking working out a problem carefully before acting. 11. robot a machine that follows instructions to move or to do a job.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Human flowchart of lining up . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER TWO**GRADE: 3 PERIOD: VI****TOPIC: Using Phones and Computers Responsibly**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Using Phones and Computers Responsibly and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	A phone is a computer in a pocket. Responsible use means asking before you pick one up, never sharing a password, and never sending a picture of a classmate. If a message asks for a photograph or a meeting, show a trusted adult the same day. Key content: 1. phone a handheld device used to call, message and run programs. 2. device a tool that uses electricity to compute, store or send information. 3. rule an agreed instruction that keeps people and devices safe. 4. safety protection from foreseeable harm, including online harm. 5. privacy keeping personal information away from people who should not see it. 6. consent freely given agreement before something is done with a person's information. 7. share to let another person see or use information. 8. report to tell a trusted adult about harm, a threat or a worrying message. 9. bully a person who repeatedly harms or frightens someone, including through a device. 10. password a secret word or code that proves you are allowed to open an account.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of The responsible-phone pledge . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER ONE**GRADE: 4 PERIOD: I****TOPIC: Breaking Problems Down**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Breaking Problems Down and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	Decomposition means breaking a big problem into smaller parts. 'Put on a school play' is too big; 'write the lines, make the hats, practise the songs' can be done. Computing problems yield to the same habit. Key content: 1. decomposition breaking a big problem into smaller parts that can be solved. 2. algorithm a finite set of steps that solves a problem or finishes a task. 3. thinking working out a problem carefully before acting. 4. project a planned computing task with a start, steps and a finished product. 5. procedure a named group of steps that can be used again. 6. step one action in a sequence of instructions. 7. pair two pupils working together on one set of instructions. 8. collaboration working with others on one computing task. 9. test trying a set of instructions with known inputs to see if they work. 10. instruction a clear step that tells a person or a machine what to do next.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Six frames of a match . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER ONE**GRADE: 4 PERIOD: II****TOPIC: If Then Choices**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in If Then Choices and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	A condition is a yes-or-no test. If it is raining, we stay in; else we go out. Computers use the same idea: if the password is right, open the file; else show an error. Selection chooses a path. Key content: 1. condition a yes-or-no test that chooses which steps to follow next. 2. selection choosing between two or more paths in an algorithm. 3. conditional a step that happens only when a test is true. 4. boolean a value that is only true or false. 5. algorithm a finite set of steps that solves a problem or finishes a task. 6. flowchart a diagram of boxes and arrows that shows the order of steps. 7. instruction a clear step that tells a person or a machine what to do next. 8. test trying a set of instructions with known inputs to see if they work. 9. thinking working out a problem carefully before acting. 10. program a set of instructions a computer can follow.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Rainy-day or dry-day line-up . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER ONE**GRADE: 4 PERIOD: III****TOPIC: How Messages Travel**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in How Messages Travel and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	A network is a set of computers that can send messages. A message is often split into packets, each with an address, and a router forwards them. Liberia's mobile networks, school wifi and even a string-and-cup line are networks with rules. Key content: 1. network a set of computers or devices that can send messages to one another. 2. message information sent from one person or device to another. 3. packet a small piece of a message sent across a network. 4. router a device that forwards packets toward their destination. 5. address a unique name or number that identifies a device or a page. 6. protocol an agreed set of rules for how messages are sent and understood. 7. wifi a wireless way for a device to join a network. 8. internet a worldwide network that lets computers share information. 9. server a computer that stores information and answers requests from other computers. 10. bandwidth how much information a connection can carry in a given time.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Packet post across the yard . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER TWO**GRADE: 4 PERIOD: IV****TOPIC: Searching for Information**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Searching for Information and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	Searching is looking through a collection until a wanted item is found. On the internet we type key words into a browser. The first result is not always the best. Ask: who wrote this, when, and what evidence do they give? Key content: 1. search looking for information by using key words. 2. searching looking through a collection until a wanted item is found. 3. source the person, page or record from which information comes. 4. evidence information that can support or challenge a claim. 5. verification checking a claim against a reliable source. 6. website a collection of pages on the internet that belong together. 7. browser a program used to look at pages on the internet. 8. link a word or picture that opens another page or file when chosen. 9. information data that has been organised so it can answer a question. 10. sort to put items into an agreed order.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Find fifteen counties, not rumours . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER TWO**GRADE: 4 PERIOD: V****TOPIC: Our Digital Footprint**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Our Digital Footprint and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	A digital footprint is the trail of information left by a person's digital activities. A nickname, a photograph, a comment and a search can last longer than a school year. Ask consent before you post about someone else. Key content: 1. digital footprint the trail of information left by a person's digital activities. 2. privacy keeping personal information away from people who should not see it. 3. consent freely given agreement before something is done with a person's information. 4. share to let another person see or use information. 5. account a named space that holds a person's files, settings and history. 6. username the public name used with a password to open an account. 7. message information sent from one person or device to another. 8. online connected to a network so messages can be sent and received. 9. citizen a person who uses computing tools with care for other people. 10. report to tell a trusted adult about harm, a threat or a worrying message. 11. digital citizen a person who uses computing tools with care for other people.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of What stays on the path . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER TWO**GRADE: 4 PERIOD: VI****TOPIC: Instructions a Computer Can Follow**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Instructions a Computer Can Follow and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	A computer follows instructions exactly, even silly ones. 'Add salt twice' means twice. Block-coding ideas start, move, turn, repeat, if can be written on cards long before a laptop runs them. Precision is kindness to the machine. Key content: 1. code instructions written in a language a computer can follow. 2. program a set of instructions a computer can follow. 3. algorithm a finite set of steps that solves a problem or finishes a task. 4. instruction a clear step that tells a person or a machine what to do next. 5. sequence steps that must be done in a fixed order. 6. flowchart a diagram of boxes and arrows that shows the order of steps. 7. pair two pupils working together on one set of instructions. 8. test trying a set of instructions with known inputs to see if they work. 9. debug to find and fix a mistake in a set of instructions. 10. thinking working out a problem carefully before acting. 11. robot a machine that follows instructions to move or to do a job.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Card commands for a paper robot . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER ONE**GRADE: 5 PERIOD: I****TOPIC: Hardware and Software**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Hardware and Software and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	Hardware is the body; software is the set of instructions. A laptop without software is a quiet box. Software without hardware has nowhere to run. An app is software written for a particular job. Key content: 1. hardware the physical parts of a computer that you can touch. 2. software the programs and instructions that tell hardware what to do. 3. processor the part of a computer that follows instructions. 4. memory the part of a computer that holds information while work is happening. 5. storage the place where a computer keeps files for later use. 6. program a set of instructions a computer can follow. 7. app a program written for a particular job on a phone, tablet or computer. 8. device a tool that uses electricity to compute, store or send information. 9. laptop a portable computer with a screen and a keyboard together. 10. desktop a computer that stays on a desk and is not meant to be carried.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Which layer failed? . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER ONE**GRADE: 5 PERIOD: II****TOPIC: Named Boxes (Variables)**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Named Boxes (Variables) and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	A variable is a named box that holds a value which can change. 'Score' can be 0, then 1, then 4. The name stays; the value changes. Using clear names 'score', not 'x' is part of writing for a person as well as a machine. Key content: 1. variable a named box that holds a value which can change. 2. name the word we give a file, a folder or a variable so we can find it again. 3. integer a whole number, with no fractional part. 4. string a sequence of letters, digits or other characters treated as text. 5. data recorded facts or numbers that can be organised and used. 6. memory the part of a computer that holds information while work is happening. 7. program a set of instructions a computer can follow. 8. instruction a clear step that tells a person or a machine what to do next. 9. test trying a set of instructions with known inputs to see if they work. 10. thinking working out a problem carefully before acting.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Score boxes for two teams . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER ONE**GRADE: 5 PERIOD: III****TOPIC: Collecting and Reading Data**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Collecting and Reading Data and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	Data are recorded facts. A tally of how many pupils walk, take a taxi or come by keke is data. A chart of that tally is information. Collect fairly, label the units, and do not invent a number you did not count. Key content: 1. data recorded facts or numbers that can be organised and used. 2. information data that has been organised so it can answer a question. 3. table rows and columns used to organise data. 4. chart a picture that shows data so a pattern is easier to see. 5. average a single number that summarises a set of numbers. 6. sort to put items into an agreed order. 7. search looking for information by using key words. 8. evidence information that can support or challenge a claim. 9. thinking working out a problem carefully before acting. 10. project a planned computing task with a start, steps and a finished product. 11. privacy keeping personal information away from people who should not see it.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of How we come to school . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER TWO**GRADE: 5 PERIOD: IV****TOPIC: Passwords and Internet Safety**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Passwords and Internet Safety and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	A password proves you are allowed to open an account. A strong password is long, unguessable and never shared not with a friend, not on a slip stuck to the screen. Cybersecurity is the set of habits that keep accounts safe. A message that shouts 'you have won' and asks for a password is a trap. Key content: 1. password a secret word or code that proves you are allowed to open an account. 2. login the steps that prove who you are before a device opens your work. 3. username the public name used with a password to open an account. 4. account a named space that holds a person's files, settings and history. 5. privacy keeping personal information away from people who should not see it. 6. cybersecurity practices that keep computers, networks and information safe. 7. malware a harmful program written to damage, steal or lock information. 8. virus a kind of malware that copies itself onto other files or devices. 9. report to tell a trusted adult about harm, a threat or a worrying message. 10. safety protection from foreseeable harm, including online harm.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Build a strong password phrase . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER TWO**GRADE: 5 PERIOD: V****TOPIC: Working Together on a Digital Task**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Working Together on a Digital Task and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	Collaboration on a digital task needs roles, comments and a shared plan. Pair programming driver and navigator is one pattern. A comment explains a step to the next person. Credit the people who did the work. Key content: 1. collaboration working with others on one computing task. 2. pair two pupils working together on one set of instructions. 3. comment a note in instructions that explains them to a person, not a machine. 4. procedure a named group of steps that can be used again. 5. project a planned computing task with a start, steps and a finished product. 6. test trying a set of instructions with known inputs to see if they work. 7. thinking working out a problem carefully before acting. 8. share to let another person see or use information. 9. file a named collection of information stored as one item. 10. folder a named place that holds files and other folders. 11. credit naming the person whose work you used.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Three roles, one poster . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER TWO**GRADE: 5 PERIOD: VI****TOPIC: Writing Instructions for a Computer**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Writing Instructions for a Computer and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	Programming is writing instructions a computer can follow. We start with an algorithm in our language, then translate it into precise steps, then test. A procedure is a named group of steps we can reuse. Parameters let the same procedure work with different data. Key content: 1. programming writing instructions a computer can follow. 2. code instructions written in a language a computer can follow. 3. program a set of instructions a computer can follow. 4. algorithm a finite set of steps that solves a problem or finishes a task. 5. procedure a named group of steps that can be used again. 6. parameter a value given to a procedure so it can do the same job with different data. 7. comment a note in instructions that explains them to a person, not a machine. 8. test trying a set of instructions with known inputs to see if they work. 9. debug to find and fix a mistake in a set of instructions. 10. sequence steps that must be done in a fixed order.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Three squares from one procedure . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER ONE**GRADE: 6 PERIOD: I****TOPIC: Hiding Details (Abstraction)**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Hiding Details (Abstraction) and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	Abstraction hides extra detail so the important idea is easier to use. A tap is an abstraction of pipes. An app icon is an abstraction of thousands of instructions. Good abstraction keeps what we need and hides what we do not. Key content: 1. abstraction hiding extra detail so the important idea is easier to use. 2. model a simpler version of a real thing used to think or to practise. 3. procedure a named group of steps that can be used again. 4. interface the part of a program a person uses, such as icons and menus. 5. user a person who gives instructions to a computer. 6. algorithm a finite set of steps that solves a problem or finishes a task. 7. thinking working out a problem carefully before acting. 8. program a set of instructions a computer can follow. 9. hardware the physical parts of a computer that you can touch. 10. icon a small picture that stands for a program, file or action.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of The tap, not the pipes . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER ONE**GRADE: 6 PERIOD: II****TOPIC: On and Off Binary**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in On and Off Binary and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	Binary uses only two digits, 0 and 1 off and on. A bit is one such digit. A byte is eight bits, enough for one ordinary character. Every picture, song and message on a computer is a pattern of ons and offs. Key content: 1. binary a way of writing information using only two digits, 0 and 1. 2. bit the smallest unit of digital information, either 0 or 1. 3. byte a group of eight bits, enough to store one ordinary character. 4. digital made of on-and-off signals that a computer can store and send. 5. on the binary state written as 1, like a lamp that is lit. 6. off the binary state written as 0, like a lamp that is dark. 7. information data that has been organised so it can answer a question. 8. storage the place where a computer keeps files for later use. 9. processor the part of a computer that follows instructions. 10. code instructions written in a language a computer can follow.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Three-bit numbers on fingers . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER ONE**GRADE: 6 PERIOD: III****TOPIC: Computing in Liberia and Africa**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Computing in Liberia and Africa and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	Computing in Liberia is already here: mobile money, radio news, market prices on a phone, school records, clinic lists. Across Africa, people write software, lay cable and teach computing. This course is an original teaching resource for Liberian classrooms, not an official syllabus transcription. Key content: 1. liberia the West African republic whose schools this course is written for. 2. africa the continent Liberia belongs to, with many computing projects of its own. 3. money value used to pay for goods and services, including mobile money. 4. phone a handheld device used to call, message and run programs. 5. radio a way of sending sound without wires, still widely used in Liberia. 6. market a place where people buy and sell, often using phones for prices and payments. 7. internet a worldwide network that lets computers share information. 8. network a set of computers or devices that can send messages to one another. 9. citizen a person who uses computing tools with care for other people. 10. project a planned computing task with a start, steps and a finished product. 11. digital citizen a person who uses computing tools with care for other people.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Map computing in our county . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER TWO**GRADE: 6 PERIOD: IV****TOPIC: Keeping Information Safe**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Keeping Information Safe and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	Cybersecurity is the set of practices that keep computers, networks and information safe. Encryption changes information so only someone with the right key can read it. Backups, updates, strong passwords and a habit of not opening strange files are the everyday tools. A locked door is useless if the key is under the mat. Key content: 1. cybersecurity practices that keep computers, networks and information safe. 2. encryption changing information so only someone with the right key can read it. 3. password a secret word or code that proves you are allowed to open an account. 4. privacy keeping personal information away from people who should not see it. 5. malware a harmful program written to damage, steal or lock information. 6. virus a kind of malware that copies itself onto other files or devices. 7. backup a second copy of files kept so work can be restored if the first copy is lost. 8. account a named space that holds a person's files, settings and history. 9. report to tell a trusted adult about harm, a threat or a worrying message. 10. safety protection from foreseeable harm, including online harm.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Backup before you click . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER TWO**GRADE: 6 PERIOD: V****TOPIC: A Computational Thinking Project**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in A Computational Thinking Project and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	Computational thinking is using decomposition, pattern, abstraction and algorithms to solve a problem. A project makes those four habits visible: choose a real problem, break it down, hide extra detail, write the steps, and test. The product can be a paper program, a poster or a performed algorithm. Key content: 1. computational using the ideas of computing steps, data and machines to solve a problem. 2. thinking working out a problem carefully before acting. 3. decomposition breaking a big problem into smaller parts that can be solved. 4. abstraction hiding extra detail so the important idea is easier to use. 5. algorithm a finite set of steps that solves a problem or finishes a task. 6. pattern something that repeats in a regular, predictable way. 7. project a planned computing task with a start, steps and a finished product. 8. test trying a set of instructions with known inputs to see if they work. 9. collaboration working with others on one computing task. 10. code instructions written in a language a computer can follow. 11. computational thinking using steps, data and machines carefully to solve a problem.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Lend a book in fewer steps . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER TWO**GRADE: 6 PERIOD: VI****TOPIC: Making a Program of Our Own**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately. Follow, write and debug an unplugged algorithm with a partner. Explain a safe, fair choice about devices and information.	Upon completion of this topic, learners will: 1. Explain the key terms in Making a Program of Our Own and use each in a sentence of your own. 2. Act out and then write an unplugged algorithm connected with this unit. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian classroom scenario. 6. Debug a broken set of steps without blaming the person who wrote them. 7. Work in a pair as driver and navigator and swap roles after five minutes. 8. Name one Liberian example of this unit's idea (market, radio, phone, office or clinic).	The last unit of Grade 6 is to make a program of our own on paper, on cards, or on a device if one is present. Plan, write, test, debug, comment, and present. A short, correct program beats a long, untested one. Credit every partner. Key content: 1. programming writing instructions a computer can follow. 2. program a set of instructions a computer can follow. 3. code instructions written in a language a computer can follow. 4. algorithm a finite set of steps that solves a problem or finishes a task. 5. procedure a named group of steps that can be used again. 6. test trying a set of instructions with known inputs to see if they work. 7. debug to find and fix a mistake in a set of instructions. 8. comment a note in instructions that explains them to a person, not a machine. 9. project a planned computing task with a start, steps and a finished product. 10. pair two pupils working together on one set of instructions.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration Pair work and debugging Classroom scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Unplugged demonstration of Show the class your program . 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and explain one choice. 6. Act the classroom scenario and agree a safe, fair response.	Materials: Chalkboard or reusable paper Word cards for the ten key terms Paper arrows, grids and a paper keyboard A class computing chart Pencils and scrap paper Optional: one teacher device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer can still complete the unit. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A four-step written algorithm Pair-debug observed against the class chart A short written response to the classroom scenario

SEMESTER ONE**GRADE: 7 PERIOD: I****TOPIC: The Computer System Hardware, Software and Users**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately in speech and writing. Write, trace and debug an algorithm, a formula or a set of steps connected with the unit on paper first, on a device when one is available. Make and justify a safe, honest and lawful choice about devices, data and people online.	Upon completion of this topic, learners will: 1. Explain the key terms in The Computer System Hardware, Software and Users and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm or set of steps connected with this unit, on paper. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step, showing every line of working, and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian school, market or office scenario. 6. Debug a broken set of steps, a formula or a short program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared record of bugs found. 8. Name one Liberian example of this unit's idea (mobile money, a radio station, a clinic record, a ministry office, a school register or a market stall).	A computer system is hardware, software and users working together. Hardware is the physical equipment; software is the set of instructions that tells the hardware what to do. Every computer follows the input process output cycle, with storage keeping data and programs for later use. Key content: 1. computer system hardware, software and users working together to turn data into information. 2. hardware the physical parts of a computer that can be seen and touched. 3. software the instructions and programs that tell the hardware what to do. 4. user the person who gives instructions to the computer and reads its results. 5. input device hardware that sends data or commands into the computer, such as a keyboard or mouse. 6. output device hardware that presents results, such as a monitor, printer or speaker. 7. processing the stage in which the processor follows instructions to change data into results. 8. storage device hardware that keeps data and programs for later, such as a hard disk or flash drive. 9. peripheral a device connected to the system unit from outside, such as a printer or scanner. 10. system unit the case that holds the processor, memory, main storage and power supply. 11. data raw facts and figures that have not yet been given meaning. 12. information data that has been processed and organised so that it means something.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration and paper trace Pair work and debugging Liberian scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of The human computer system with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, paper grids and a paper keyboard A class computing chart Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's phone, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer lab can still complete the unit on paper, and a class with one shared device can extend it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written algorithm, trace table or worked conversion Pair-debug observed against the class chart A short written response to the Liberian scenario

SEMESTER ONE**GRADE: 7 PERIOD: II****TOPIC: Generations and Types of Computers**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately in speech and writing. Write, trace and debug an algorithm, a formula or a set of steps connected with the unit on paper first, on a device when one is available. Make and justify a safe, honest and lawful choice about devices, data and people online.	Upon completion of this topic, learners will: 1. Explain the key terms in Generations and Types of Computers and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm or set of steps connected with this unit, on paper. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step, showing every line of working, and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian school, market or office scenario. 6. Debug a broken set of steps, a formula or a short program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared record of bugs found. 8. Name one Liberian example of this unit's idea (mobile money, a radio station, a clinic record, a ministry office, a school register or a market stall).	The generations of computers are named after the switching technology inside them: vacuum tubes, transistors, integrated circuits and microprocessors, with today's connected and intelligent devices often called a fifth generation. Types of computer supercomputer, mainframe, minicomputer, microcomputer (desktop, laptop, tablet, smartphone) and embedded computers differ in size, speed, cost and the work they are built for. Key content: 1. generation a stage in computer history named after the main switching technology used inside the machine. 2. vacuum tube a glass valve used as a switch in first-generation computers; large, hot and unreliable. 3. transistor a small electronic switch that replaced the vacuum tube in the second generation. 4. integrated circuit many transistors built onto one small piece of silicon, used from the third generation. 5. microprocessor a complete processor on a single chip, the heart of fourth-generation and later computers. 6. mainframe a large, powerful computer that serves many users at once, used by banks and governments. 7. supercomputer the fastest type of computer, used for weather forecasting and scientific research. 8. microcomputer a computer built around one microprocessor and used by one person at a time, such as a desktop or laptop. 9. laptop a portable microcomputer with the screen, keyboard and battery in one folding case.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration and paper trace Pair work and debugging Liberian scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of The generations timeline walk with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, paper grids and a paper keyboard A class computing chart Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's phone, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer lab can still complete the unit on paper, and a class with one shared device can extend it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written algorithm, trace table or worked conversion Pair-debug observed against the class chart A short written response to the Liberian scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		10. smartphone a handheld computer that also makes calls, with a touchscreen, camera and internet connection. 11. embedded computer a small computer built into another machine to control it, such as in a car or a washing machine. 12. artificial intelligence software that learns patterns from data to perform tasks that normally need human judgement.				

SEMESTER ONE**GRADE: 7 PERIOD: III****TOPIC: Keyboard Skills and Word Processing**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately in speech and writing. Write, trace and debug an algorithm, a formula or a set of steps connected with the unit on paper first, on a device when one is available. Make and justify a safe, honest and lawful choice about devices, data and people online.	Upon completion of this topic, learners will: 1. Explain the key terms in Keyboard Skills and Word Processing and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm or set of steps connected with this unit, on paper. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step, showing every line of working, and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian school, market or office scenario. 6. Debug a broken set of steps, a formula or a short program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared record of bugs found. 8. Name one Liberian example of this unit's idea (mobile money, a radio station, a clinic record, a ministry office, a school register or a market stall).	A word processor is software for creating, editing, formatting, saving and printing documents. Touch-typing starts from the home row (A S D F and J K L ;) with the thumbs on the space bar. Editing changes the words; formatting changes how they look font, size, bold, italic, underline, alignment and spacing. Key content: 1. word processor software for creating, editing, formatting, saving and printing documents. 2. home row the middle row of letter keys A S D F and J K L ; where the fingers rest when touch-typing. 3. touch-typing typing with all fingers without looking at the keys. 4. cursor the blinking line or pointer that shows where the next character will appear. 5. editing changing the words of a document: inserting, deleting, cutting, copying and pasting. 6. formatting changing how text looks: font, size, bold, italic, underline, colour and alignment. 7. font a named design of letters, such as Times New Roman or Arial. 8. alignment how lines sit against the margins: left, centred, right or justified. 9. paragraph a block of related sentences, ended in a word processor by pressing Enter. 10. margin the blank space between the text and the edge of the page. 11. spell checker a tool that underlines words it does not recognise so the writer can check them. 12. save to copy the document from memory to storage under a file	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration and paper trace Pair work and debugging Liberian scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of Home-row drill on a paper keyboard with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, paper grids and a paper keyboard A class computing chart Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's phone, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer lab can still complete the unit on paper, and a class with one shared device can extend it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written algorithm, trace table or worked conversion Pair-debug observed against the class chart A short written response to the Liberian scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		name so it is not lost.				

SEMESTER TWO**GRADE: 7 PERIOD: IV****TOPIC: The Operating System and Managing Files**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately in speech and writing. Write, trace and debug an algorithm, a formula or a set of steps connected with the unit on paper first, on a device when one is available. Make and justify a safe, honest and lawful choice about devices, data and people online.	Upon completion of this topic, learners will: 1. Explain the key terms in The Operating System and Managing Files and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm or set of steps connected with this unit, on paper. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step, showing every line of working, and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian school, market or office scenario. 6. Debug a broken set of steps, a formula or a short program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared record of bugs found. 8. Name one Liberian example of this unit's idea (mobile money, a radio station, a clinic record, a ministry office, a school register or a market stall).	An operating system (such as Windows, Android, Linux or macOS) manages the hardware, runs programs and organises files. A file is a named collection of data; a folder holds files and other folders; a path names the route from the drive to the file. Managing files means naming, saving, copying, moving, renaming and deleting them carefully. Key content: 1. operating system the software that manages the hardware, runs the other programs and organises files. 2. boot to start a computer by loading the operating system from storage into memory. 3. desktop the first screen the operating system shows, with icons, a taskbar and a background. 4. icon a small picture that stands for a file, folder, program or drive. 5. window a framed area of the screen in which one program or folder is shown. 6. file a named collection of data stored as one unit, such as a letter, a photo or a program. 7. folder a named container that holds files and other folders, also called a directory. 8. file name the name given to a file so that it can be found again. 9. extension the letters after the last dot in a file name that show the file's type, such as .docx or .jpg. 10. path the route to a file through the drive and folders, such as D:\School\Grade7\notice.docx. 11. shut down to close all programs and switch off safely through the operating system. 12. recycle bin the folder that holds deleted files until it is emptied, so a mistake can be undone.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration and paper trace Pair work and debugging Liberian scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of The paper filing cabinet with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, paper grids and a paper keyboard A class computing chart Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's phone, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer lab can still complete the unit on paper, and a class with one shared device can extend it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written algorithm, trace table or worked conversion Pair-debug observed against the class chart A short written response to the Liberian scenario

SEMESTER TWO**GRADE: 7 PERIOD: V****TOPIC: Health, Safety and Care of ICT Tools**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately in speech and writing. Write, trace and debug an algorithm, a formula or a set of steps connected with the unit on paper first, on a device when one is available. Make and justify a safe, honest and lawful choice about devices, data and people online.	Upon completion of this topic, learners will: 1. Explain the key terms in Health, Safety and Care of ICT Tools and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm or set of steps connected with this unit, on paper. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step, showing every line of working, and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian school, market or office scenario. 6. Debug a broken set of steps, a formula or a short program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared record of bugs found. 8. Name one Liberian example of this unit's idea (mobile money, a radio station, a clinic record, a ministry office, a school register or a market stall).	Safe computing protects the user, the machine and the work. Users need good posture, a screen at eye level, breaks every 20 minutes and clean, dry hands. Machines need steady power through a surge protector, protection from dust, heat, water and falls, and careful shutdown. Work needs regular saving and a backup on a second drive. Dead devices are e-waste and must be handled and disposed of safely. Key content: 1. ergonomics arranging equipment and posture so that work fits the human body and does no harm. 2. repetitive strain injury pain in the hands, wrists or arms caused by the same small movement repeated for hours. 3. eye strain tired, sore or dry eyes caused by staring at a screen without breaks or in bad light. 4. posture the way the body is held while sitting and working: feet flat, back supported, wrists straight. 5. surge protector a device that blocks sudden jumps in voltage so they do not damage the computer. 6. uninterruptible power supply a battery unit that keeps a computer running for a few minutes when the power fails, so it can be shut down safely. 7. ventilation the flow of air around a machine that carries heat away and prevents overheating. 8. backup a second copy of files kept on a different drive or device in case the first is lost. 9. e-waste discarded electrical and electronic devices, batteries and cables that must be disposed of safely.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration and paper trace Pair work and debugging Liberian scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of The workstation safety audit with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, paper grids and a paper keyboard A class computing chart Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's phone, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer lab can still complete the unit on paper, and a class with one shared device can extend it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written algorithm, trace table or worked conversion Pair-debug observed against the class chart A short written response to the Liberian scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		10. dust cover a cloth or plastic cover placed over a machine when it is not in use to keep dust and insects out. 11. 20-20-20 rule every 20 minutes, look at something 20 feet (about 6 metres) away for 20 seconds. 12. trip hazard a cable or object on the floor that a person can catch a foot on and fall over.				

SEMESTER TWO**GRADE: 7 PERIOD: VI****TOPIC: Algorithms, Flowcharts and Trace Tables**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately in speech and writing. Write, trace and debug an algorithm, a formula or a set of steps connected with the unit on paper first, on a device when one is available. Make and justify a safe, honest and lawful choice about devices, data and people online.	Upon completion of this topic, learners will: 1. Explain the key terms in Algorithms, Flowcharts and Trace Tables and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm or set of steps connected with this unit, on paper. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step, showing every line of working, and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian school, market or office scenario. 6. Debug a broken set of steps, a formula or a short program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared record of bugs found. 8. Name one Liberian example of this unit's idea (mobile money, a radio station, a clinic record, a ministry office, a school register or a market stall).	An algorithm is a finite, ordered set of steps that solves a problem. Pseudocode writes it in numbered plain English; a flowchart draws it with a terminal (oval), process (rectangle), decision (diamond), input/output (parallelogram) and arrows. Sequence, selection and iteration are the three control structures, and a trace table is how an algorithm is tested by hand. Key content: 1. algorithm a finite, ordered set of steps that solves a problem or completes a task. 2. pseudocode an algorithm written in numbered plain-English steps that are precise enough to follow exactly. 3. flowchart a diagram of an algorithm using standard shapes joined by arrows. 4. sequence steps carried out one after another in order. 5. selection a decision that chooses between paths depending on a yes-or-no condition. 6. iteration repeating a step or group of steps, also called a loop. 7. condition a question with a yes-or-no answer that controls a selection or a loop. 8. variable a named place that holds a value which can change while the algorithm runs. 9. trace table a table with one column per variable and one row per step, used to test an algorithm by hand. 10. decision symbol the diamond in a flowchart that holds a condition with a yes arrow and a no arrow. 11. process symbol the rectangle in a flowchart that holds a calculation or an action.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration and paper trace Pair work and debugging Liberian scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of The market-total trace table with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, paper grids and a paper keyboard A class computing chart Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's phone, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer lab can still complete the unit on paper, and a class with one shared device can extend it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written algorithm, trace table or worked conversion Pair-debug observed against the class chart A short written response to the Liberian scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		12. terminal symbol the oval in a flowchart that marks Start and Stop.				

SEMESTER ONE**GRADE: 8 PERIOD: I****TOPIC: Inside the Computer CPU, Memory and Storage**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately in speech and writing. Write, trace and debug an algorithm, a formula or a set of steps connected with the unit on paper first, on a device when one is available. Make and justify a safe, honest and lawful choice about devices, data and people online.	Upon completion of this topic, learners will: 1. Explain the key terms in Inside the Computer CPU, Memory and Storage and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm or set of steps connected with this unit, on paper. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step, showing every line of working, and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian school, market or office scenario. 6. Debug a broken set of steps, a formula or a short program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared record of bugs found. 8. Name one Liberian example of this unit's idea (mobile money, a radio station, a clinic record, a ministry office, a school register or a market stall).	The central processing unit (CPU) fetches, decodes and executes instructions using its control unit and arithmetic logic unit. RAM is fast, temporary working memory; ROM holds permanent start-up instructions; secondary storage keeps files when the power is off. Storage is measured in bytes: 1 KB 1,000 bytes, 1 MB 1,000 KB, 1 GB 1,000 MB, 1 TB 1,000 GB. Key content: 1. central processing unit the chip that carries out instructions; the 'brain' of the computer, also called the CPU or processor. 2. control unit the part of the CPU that fetches each instruction, decodes it and directs the other parts. 3. arithmetic logic unit the part of the CPU that does arithmetic (add, subtract) and comparisons (greater than, equal to). 4. fetch decode execute cycle the repeated steps by which the CPU gets an instruction from memory, works out what it means and carries it out. 5. RAM random-access memory: fast working memory that holds running programs and data, and is emptied when the power goes off. 6. ROM read-only memory: permanent memory holding the start-up instructions the computer needs to boot. 7. secondary storage devices such as hard disks, solid-state drives and flash drives that keep data when the power is off. 8. hard disk drive a storage device that records data magnetically on spinning platters. 9. solid-state drive a storage device with no moving parts that keeps data in flash memory chips.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration and paper trace Pair work and debugging Liberian scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of The fetch decode execute relay with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, paper grids and a paper keyboard A class computing chart Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's phone, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer lab can still complete the unit on paper, and a class with one shared device can extend it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written algorithm, trace table or worked conversion Pair-debug observed against the class chart A short written response to the Liberian scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		10. byte the unit of storage that holds one character; eight bits. 11. gigabyte about one thousand million bytes (1,000 MB); the usual unit for a phone's storage. 12. clock speed the number of cycles the CPU can run each second, measured in gigahertz (GHz).				

SEMESTER ONE**GRADE: 8 PERIOD: II****TOPIC: Spreadsheets Cells, Formulas and Functions**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately in speech and writing. Write, trace and debug an algorithm, a formula or a set of steps connected with the unit on paper first, on a device when one is available. Make and justify a safe, honest and lawful choice about devices, data and people online.	Upon completion of this topic, learners will: 1. Explain the key terms in Spreadsheets Cells, Formulas and Functions and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm or set of steps connected with this unit, on paper. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step, showing every line of working, and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian school, market or office scenario. 6. Debug a broken set of steps, a formula or a short program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared record of bugs found. 8. Name one Liberian example of this unit's idea (mobile money, a radio station, a clinic record, a ministry office, a school register or a market stall).	A spreadsheet is a grid of cells addressed by column letter and row number. A formula begins with = and calculates from other cells (=B2*C2); functions such as SUM, AVERAGE, MAX, MIN and COUNT work on a range (=SUM(D2:D6)). When a value changes, every formula that uses it recalculates automatically. Key content: 1. spreadsheet a program that stores numbers, text and formulas in a grid of cells and recalculates them automatically. 2. cell one box in the grid, named by its column letter and row number, such as B3. 3. column a vertical line of cells named by a letter: A, B, C . 4. row a horizontal line of cells named by a number: 1, 2, 3 . 5. formula an instruction beginning with = that calculates a value from other cells, such as =B2*C2. 6. function a ready-made formula with a name, such as SUM, AVERAGE, MAX, MIN or COUNT. 7. range a block of cells named by its first and last cell, such as B2:B6. 8. SUM the function that adds every number in a range. 9. AVERAGE the function that adds the numbers in a range and divides by how many there are. 10. recalculate to work out every formula again after a cell changes. 11. fill down to copy a formula down a column so its cell references move with it. 12. chart a picture of the numbers in a range, such as a bar chart or a pie chart.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration and paper trace Pair work and debugging Liberian scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of The paper market ledger with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, paper grids and a paper keyboard A class computing chart Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's phone, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer lab can still complete the unit on paper, and a class with one shared device can extend it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written algorithm, trace table or worked conversion Pair-debug observed against the class chart A short written response to the Liberian scenario

SEMESTER ONE**GRADE: 8 PERIOD: III****TOPIC: Computer Networks LAN, WAN and How Data Travels**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately in speech and writing. Write, trace and debug an algorithm, a formula or a set of steps connected with the unit on paper first, on a device when one is available. Make and justify a safe, honest and lawful choice about devices, data and people online.	Upon completion of this topic, learners will: 1. Explain the key terms in Computer Networks LAN, WAN and How Data Travels and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm or set of steps connected with this unit, on paper. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step, showing every line of working, and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian school, market or office scenario. 6. Debug a broken set of steps, a formula or a short program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared record of bugs found. 8. Name one Liberian example of this unit's idea (mobile money, a radio station, a clinic record, a ministry office, a school register or a market stall).	A network connects computers so they can share data and devices. A LAN covers one site; a WAN links sites over long distances the internet is the largest WAN. Data travels as packets with addresses; switches connect devices within a LAN, routers pass packets between networks, and a modem connects to the line. Media include copper cable, fibre-optic cable and wireless radio (Wi-Fi, mobile data). Key content: 1. network two or more computers connected so that they can share data, devices and connections. 2. local area network a network covering one building or site, such as a school or an office; a LAN. 3. wide area network a network that joins LANs over long distances, across towns and countries; a WAN. 4. packet a small block of data with the sender's and receiver's addresses, into which a message is split for sending. 5. switch a device inside a LAN that passes each packet only to the device it is addressed to. 6. router a device that passes packets between different networks and chooses the best route. 7. modem a device that converts computer signals to and from the form the telephone, cable or fibre line carries. 8. Wi-Fi a wireless way of joining a LAN using radio signals. 9. fibre-optic cable a cable that carries data as pulses of light through glass fibres, very fast over long distances. 10. bandwidth the amount of data a connection can carry each second, measured in bits per second.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration and paper trace Pair work and debugging Liberian scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of The classroom packet network with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, paper grids and a paper keyboard A class computing chart Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's phone, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer lab can still complete the unit on paper, and a class with one shared device can extend it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written algorithm, trace table or worked conversion Pair-debug observed against the class chart A short written response to the Liberian scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		11. IP address the number that identifies a device on a network so packets can reach it. 12. server a computer that provides a service files, web pages, email to other computers on a network.				

SEMESTER TWO**GRADE: 8 PERIOD: IV****TOPIC: The Internet, the Web and Email**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately in speech and writing. Write, trace and debug an algorithm, a formula or a set of steps connected with the unit on paper first, on a device when one is available. Make and justify a safe, honest and lawful choice about devices, data and people online.	Upon completion of this topic, learners will: 1. Explain the key terms in The Internet, the Web and Email and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm or set of steps connected with this unit, on paper. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step, showing every line of working, and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian school, market or office scenario. 6. Debug a broken set of steps, a formula or a short program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared record of bugs found. 8. Name one Liberian example of this unit's idea (mobile money, a radio station, a clinic record, a ministry office, a school register or a market stall).	The internet is the global network of networks; the web, email and messaging are services that use it. A browser shows web pages found by their URLs; a search engine is a website that indexes other pages. Email carries messages and attachments between addresses. Every page has an author with a purpose, so readers check who wrote it, when, and whether other sources agree. Key content: 1. internet the world-wide network of networks that connects computers and phones in every country. 2. World Wide Web the service on the internet made of linked pages that a browser displays. 3. browser the program that fetches and displays web pages, such as Chrome, Firefox or Opera. 4. URL the address of a web page, such as https://www.moe.gov.lr/exams. 5. hyperlink text or a picture on a page that opens another page when clicked. 6. search engine a website that keeps an index of other pages and lists those that match the words you type. 7. email electronic mail: a message sent over the internet from one address to another. 8. attachment a file sent along with an email, such as a document or a photograph. 9. domain name the part of a URL that names the site and its type, such as moe.gov.lr. 10. source where a piece of information comes from the person or organisation that published it.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration and paper trace Pair work and debugging Liberian scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of The paper web and the source check with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, paper grids and a paper keyboard A class computing chart Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's phone, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer lab can still complete the unit on paper, and a class with one shared device can extend it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written algorithm, trace table or worked conversion Pair-debug observed against the class chart A short written response to the Liberian scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		11. misinformation false or misleading information, whether spread deliberately or by mistake. 12. HTTPS the secure version of the web's transfer rules, shown by a padlock, that encrypts the page on its way.				

SEMESTER TWO**GRADE: 8 PERIOD: V****TOPIC: Information Security and Digital Citizenship**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately in speech and writing. Write, trace and debug an algorithm, a formula or a set of steps connected with the unit on paper first, on a device when one is available. Make and justify a safe, honest and lawful choice about devices, data and people online.	Upon completion of this topic, learners will: 1. Explain the key terms in Information Security and Digital Citizenship and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm or set of steps connected with this unit, on paper. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step, showing every line of working, and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian school, market or office scenario. 6. Debug a broken set of steps, a formula or a short program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared record of bugs found. 8. Name one Liberian example of this unit's idea (mobile money, a radio station, a clinic record, a ministry office, a school register or a market stall).	Information security protects data from being read, changed or destroyed by people who should not. Strong passwords, two-factor authentication, updates, backups and antivirus software are the main defences; phishing, malware and social engineering are the main attacks. Digital citizenship means behaving online with the same respect, honesty and consent as offline, and leaving a digital footprint you can stand behind. Key content: 1. information security protecting data so that it stays private, correct and available to the people who should have it. 2. password a secret string of characters that proves you are the owner of an account. 3. two-factor authentication proving who you are with two different things, such as a password and a code sent to your phone. 4. phishing a fake message or website that pretends to be a trusted organisation to trick you into giving a password, PIN or money. 5. malware malicious software viruses, worms, ransomware, spyware that damages, steals or locks data. 6. social engineering tricking a person, rather than a machine, into giving access or information. 7. PIN a personal identification number, the secret code that protects a mobile-money wallet or a bank card. 8. cyberbullying using messages, posts or images to threaten, shame or harass a person. 9. digital footprint the trail of posts, photos, searches and messages a person leaves online, much of which can never be deleted.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration and paper trace Pair work and debugging Liberian scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of The scam-or-safe sorting court with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, paper grids and a paper keyboard A class computing chart Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's phone, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer lab can still complete the unit on paper, and a class with one shared device can extend it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written algorithm, trace table or worked conversion Pair-debug observed against the class chart A short written response to the Liberian scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		10. consent clear permission, freely given, before a person's image, words or details are shared. 11. privacy settings the controls on an account that decide who can see your posts, photos and details. 12. update a new version of software that fixes security holes and bugs; installing it closes doors attackers use.				

SEMESTER TWO**GRADE: 8 PERIOD: VI****TOPIC: First Programs Variables, Input and Output**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately in speech and writing. Write, trace and debug an algorithm, a formula or a set of steps connected with the unit on paper first, on a device when one is available. Make and justify a safe, honest and lawful choice about devices, data and people online.	Upon completion of this topic, learners will: 1. Explain the key terms in First Programs Variables, Input and Output and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm or set of steps connected with this unit, on paper. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step, showing every line of working, and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian school, market or office scenario. 6. Debug a broken set of steps, a formula or a short program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared record of bugs found. 8. Name one Liberian example of this unit's idea (mobile money, a radio station, a clinic record, a ministry office, a school register or a market stall).	A program is an algorithm written in a programming language. Variables hold values that can change; assignment (name = value) stores a value; input reads a value from the user and output displays a result. Every value has a data type integer, float, string or boolean and arithmetic follows the usual order of operations. A program is tested by tracing it on paper with sample inputs. Key content: 1. program an algorithm written in a programming language so that a computer can run it. 2. programming language a language with strict rules, such as Python or Scratch, in which programs are written. 3. variable a named place in memory that holds a value which can change while the program runs. 4. assignment storing a value in a variable, written name = value, as in total = 0. 5. input taking a value from the user, usually typed at the keyboard, and storing it in a variable. 6. output displaying a value or message to the user, as in print(total). 7. data type the kind of value a variable holds: integer, float, string or boolean. 8. integer a whole number with no decimal part, such as 3 or -12. 9. float a number with a decimal part, such as 37.5 or 2.0. 10. string a piece of text inside quotation marks, such as "Hello" or "Grade 8". 11. syntax error a mistake in the spelling or punctuation of the code, so the program cannot run at all.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration and paper trace Pair work and debugging Liberian scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of The change calculator, traced by hand with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, paper grids and a paper keyboard A class computing chart Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's phone, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer lab can still complete the unit on paper, and a class with one shared device can extend it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written algorithm, trace table or worked conversion Pair-debug observed against the class chart A short written response to the Liberian scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		12. logic error a mistake in the plan of the program, so it runs but gives the wrong answer.				

SEMESTER ONE**GRADE: 9 PERIOD: I****TOPIC: Presentation Software and Communicating Ideas**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately in speech and writing. Write, trace and debug an algorithm, a formula or a set of steps connected with the unit on paper first, on a device when one is available. Make and justify a safe, honest and lawful choice about devices, data and people online.	Upon completion of this topic, learners will: 1. Explain the key terms in Presentation Software and Communicating Ideas and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm or set of steps connected with this unit, on paper. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step, showing every line of working, and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian school, market or office scenario. 6. Debug a broken set of steps, a formula or a short program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared record of bugs found. 8. Name one Liberian example of this unit's idea (mobile money, a radio station, a clinic record, a ministry office, a school register or a market stall).	Presentation software arranges a talk as a sequence of slides with titles, short bullet points, images and charts. A storyboard plans the slides on paper first. Good design keeps each slide to one idea, uses large readable text with strong contrast and few words, and treats transitions and animations lightly. The presenter speaks from notes; the slides support the speech. Key content: 1. presentation software a program that builds a sequence of slides to support a talk, such as PowerPoint, Impress or Google Slides. 2. slide one screen of a presentation, usually holding a title and one idea. 3. storyboard a paper plan of a presentation showing each slide's title, main point and picture in order. 4. bullet point a short phrase marked with a dot, used to list ideas on a slide. 5. slide layout the arrangement of title, text and picture placeholders on a slide. 6. transition the visual effect used when one slide changes to the next. 7. animation movement applied to an object on a slide, such as a bullet appearing on a click. 8. speaker notes the presenter's own words for each slide, seen by the presenter and not by the audience. 9. contrast the difference between the colour of text and its background; high contrast is easy to read. 10. audience the people a presentation is made for, whose needs decide its words, pictures and length.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration and paper trace Pair work and debugging Liberian scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of The six-slide paper storyboard with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, paper grids and a paper keyboard A class computing chart Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's phone, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer lab can still complete the unit on paper, and a class with one shared device can extend it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written algorithm, trace table or worked conversion Pair-debug observed against the class chart A short written response to the Liberian scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		11. 6-by-6 guide a rule of thumb: no more than six bullet points on a slide and six words in each. 12. handout a printed copy of the slides or a summary given to the audience.				

SEMESTER ONE**GRADE: 9 PERIOD: II****TOPIC: Programming Selection and Repetition**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately in speech and writing. Write, trace and debug an algorithm, a formula or a set of steps connected with the unit on paper first, on a device when one is available. Make and justify a safe, honest and lawful choice about devices, data and people online.	Upon completion of this topic, learners will: 1. Explain the key terms in Programming Selection and Repetition and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm or set of steps connected with this unit, on paper. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step, showing every line of working, and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian school, market or office scenario. 6. Debug a broken set of steps, a formula or a short program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared record of bugs found. 8. Name one Liberian example of this unit's idea (mobile money, a radio station, a clinic record, a ministry office, a school register or a market stall).	Selection uses if, elif and else with conditions built from the comparison operators (equal ==, not equal !=, less than, greater than, at most, at least) and the logical operators and, or, not. Iteration uses a while loop (repeat while a condition is true) or a for loop (repeat a fixed number of times or once per item). Counters and accumulators are updated inside the loop, and a trace table checks every pass. Key content: 1. selection choosing which lines to run using if, elif and else with a condition. 2. condition an expression that is either True or False, such as mark >= 50. 3. comparison operator a symbol that compares two values: == equal, != not equal, less than, greater than, less than or equal, greater than or equal. 4. logical operator and, or and not, which combine or reverse conditions. 5. else the branch that runs when no condition above it was true. 6. elif short for 'else if': another condition tested only if the ones above were false. 7. iteration repeating lines of code; also called looping. 8. while loop a loop that repeats its body while its condition remains true. 9. for loop a loop that repeats a fixed number of times or once for each item, such as for i in range(5). 10. counter a variable that counts how many times something has happened, increased by 1 each pass. 11. accumulator a variable that builds up a total, increased by each new value in the loop.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration and paper trace Pair work and debugging Liberian scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of The pass-fail-credit grader with a counter with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, paper grids and a paper keyboard A class computing chart Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's phone, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer lab can still complete the unit on paper, and a class with one shared device can extend it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written algorithm, trace table or worked conversion Pair-debug observed against the class chart A short written response to the Liberian scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		12. infinite loop a loop whose condition never becomes false, so it never stops.				

SEMESTER ONE**GRADE: 9 PERIOD: III****TOPIC: Data Representation Text, Images and Sound**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately in speech and writing. Write, trace and debug an algorithm, a formula or a set of steps connected with the unit on paper first, on a device when one is available. Make and justify a safe, honest and lawful choice about devices, data and people online.	Upon completion of this topic, learners will: 1. Explain the key terms in Data Representation Text, Images and Sound and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm or set of steps connected with this unit, on paper. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step, showing every line of working, and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian school, market or office scenario. 6. Debug a broken set of steps, a formula or a short program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared record of bugs found. 8. Name one Liberian example of this unit's idea (mobile money, a radio station, a clinic record, a ministry office, a school register or a market stall).	Computers store text as character codes (ASCII uses one byte per character; Unicode covers all scripts), images as grids of pixels whose colours are numbers (file size width height colour depth), and sound as a stream of samples taken thousands of times per second. Larger resolution, colour depth or sample rate means better quality and bigger files; compression reduces size for storage and sending. Key content: 1. character set the agreed table that gives every letter, digit and symbol a number code. 2. ASCII an early character set that codes 128 English characters using one byte each; A is 65, a is 97. 3. Unicode the modern character set that gives a code to every character in every script, including Vai and Arabic. 4. pixel one tiny square of colour in a digital image; short for picture element. 5. resolution the number of pixels across and down an image, such as 1,200 800. 6. colour depth the number of bits used for each pixel's colour; 1 bit gives two colours, 8 bits give 256, 24 bits give millions. 7. bitmap an image stored as a grid of pixel colours, such as a photograph. 8. sample one measurement of a sound wave's height taken at an instant. 9. sample rate how many samples are taken each second, such as 44,100 for CD quality. 10. file size the number of bytes a file occupies in storage. 11. compression reducing a file's size, either without losing detail (lossless) or by discarding detail	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration and paper trace Pair work and debugging Liberian scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of Encoding a word and a picture on paper with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, paper grids and a paper keyboard A class computing chart Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's phone, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer lab can still complete the unit on paper, and a class with one shared device can extend it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written algorithm, trace table or worked conversion Pair-debug observed against the class chart A short written response to the Liberian scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		(lossy). 12. metadata data about data stored with a file, such as an image's size, date and the camera used.				

SEMESTER TWO**GRADE: 9 PERIOD: IV****TOPIC: Databases Tables, Records and Fields**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately in speech and writing. Write, trace and debug an algorithm, a formula or a set of steps connected with the unit on paper first, on a device when one is available. Make and justify a safe, honest and lawful choice about devices, data and people online.	Upon completion of this topic, learners will: 1. Explain the key terms in Databases Tables, Records and Fields and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm or set of steps connected with this unit, on paper. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step, showing every line of working, and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian school, market or office scenario. 6. Debug a broken set of steps, a formula or a short program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared record of bugs found. 8. Name one Liberian example of this unit's idea (mobile money, a radio station, a clinic record, a ministry office, a school register or a market stall).	A database stores data in tables of records (rows) and fields (columns). A primary key uniquely identifies each record. Data types for fields (text, number, date, yes/no) and validation rules keep the data clean. Queries search and sort the data using conditions; reports present the results. Compared with paper, a database finds, sorts and updates records quickly and keeps one copy of the truth. Key content: 1. database an organised collection of related data stored so it can be searched, sorted and updated easily. 2. table a set of records about one kind of thing, arranged in rows and columns. 3. record one row of a table all the fields about one person, item or event. 4. field one column of a table one piece of data kept for every record, such as Surname. 5. primary key the field whose value is different for every record, so each record can be found without confusion. 6. data type the kind of value a field holds: text, number, date, currency or yes/no. 7. validation a rule that checks data as it is entered and rejects values that cannot be right. 8. query a question put to the database using conditions, whose answer is a table of matching records. 9. sort to arrange records in order of a field, ascending or descending. 10. report a printed or displayed presentation of the results of a query. 11. duplicate a record that repeats one already stored, usually a	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration and paper trace Pair work and debugging Liberian scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of The card-index school database with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, paper grids and a paper keyboard A class computing chart Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's phone, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer lab can still complete the unit on paper, and a class with one shared device can extend it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written algorithm, trace table or worked conversion Pair-debug observed against the class chart A short written response to the Liberian scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		sign of a missing or ignored primary key. 12. flat file a database made of a single table.				

SEMESTER TWO**GRADE: 9 PERIOD: V****TOPIC: Web Pages HTML Structure and Links**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately in speech and writing. Write, trace and debug an algorithm, a formula or a set of steps connected with the unit on paper first, on a device when one is available. Make and justify a safe, honest and lawful choice about devices, data and people online.	Upon completion of this topic, learners will: 1. Explain the key terms in Web Pages HTML Structure and Links and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm or set of steps connected with this unit, on paper. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step, showing every line of working, and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian school, market or office scenario. 6. Debug a broken set of steps, a formula or a short program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared record of bugs found. 8. Name one Liberian example of this unit's idea (mobile money, a radio station, a clinic record, a ministry office, a school register or a market stall).	HTML marks up the structure of a web page with tags: html, head, title and body; headings h1 to h6; paragraphs p; lists ul and li; links a with an href attribute; and images img with src and alt attributes. Most tags open and close; attributes add detail inside the opening tag. The browser reads the HTML and draws the page; CSS (met in Grade 12) controls how it looks. Key content: 1. HTML HyperText Markup Language, the language that marks up the structure of a web page. 2. tag a marker written in angle brackets, such as the p tag, that tells the browser what a part of the page is. 3. element an opening tag, its content and its closing tag together, such as a p tag, the word Hello and the closing p tag. 4. attribute extra information inside an opening tag, such as href="index.html" inside an a tag. 5. heading a title on the page, marked with the h1 tag for the largest down to h6 for the smallest. 6. paragraph a block of text marked with an opening p tag and a closing p tag. 7. hyperlink text or an image marked with an a tag whose href attribute names another page, opened when clicked. 8. image tag the img tag, whose src attribute names the picture file and whose alt attribute describes it for readers who cannot see it. 9. list items marked with li tags inside a ul tag for bullets or an ol tag for numbers. 10. head the part of an HTML file, inside the head tag, that holds the page's title and settings and is not shown on the page.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration and paper trace Pair work and debugging Liberian scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of Writing a school page by hand and rendering it in pairs with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, paper grids and a paper keyboard A class computing chart Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's phone, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer lab can still complete the unit on paper, and a class with one shared device can extend it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written algorithm, trace table or worked conversion Pair-debug observed against the class chart A short written response to the Liberian scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		11. body the part of an HTML file, inside the body tag, that holds everything shown on the page. 12. web server a computer that stores web pages and sends them to browsers that ask for them.				

SEMESTER TWO**GRADE: 9 PERIOD: VI****TOPIC: Emerging Technology, Careers and a Computing Project**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary accurately in speech and writing. Write, trace and debug an algorithm, a formula or a set of steps connected with the unit on paper first, on a device when one is available. Make and justify a safe, honest and lawful choice about devices, data and people online.	Upon completion of this topic, learners will: 1. Explain the key terms in Emerging Technology, Careers and a Computing Project and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm or set of steps connected with this unit, on paper. 3. Sort examples into the unit's two groups and give a reason for each choice. 4. Trace a worked example step by step, showing every line of working, and state the answer in a full sentence. 5. Apply the unit's ideas to a fictional Liberian school, market or office scenario. 6. Debug a broken set of steps, a formula or a short program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared record of bugs found. 8. Name one Liberian example of this unit's idea (mobile money, a radio station, a clinic record, a ministry office, a school register or a market stall).	Emerging technologies artificial intelligence, robotics and drones, the Internet of Things, cloud services, mobile money and e-government bring benefits, risks and ethical questions, and each creates new careers. A computing project follows a cycle: define the problem, plan a solution, build it (on paper or a device), test it with real data, and present it with credit to sources and partners. Key content: 1. artificial intelligence software that learns patterns from large amounts of data to make predictions or decisions, such as recognising a face or a crop disease. 2. machine learning the branch of artificial intelligence in which a program improves by training on examples rather than being given every rule. 3. training data the examples a machine-learning program learns from; biased or poor data gives biased or poor results. 4. robotics machines that sense, decide and act in the physical world, such as a drone or a factory arm. 5. drone a small unpiloted aircraft used for mapping, photography and delivering light loads. 6. Internet of Things everyday objects meters, pumps, sensors fitted with small computers and connected to a network. 7. cloud computing using storage and programs on distant servers over the internet instead of on your own machine. 8. e-government public services delivered online, such as applying for a document or checking examination results. 9. automation using machines or software to do work that people used	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Unplugged demonstration and paper trace Pair work and debugging Liberian scenario Computing fair check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of The school computing project, define to present with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, paper grids and a paper keyboard A class computing chart Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's phone, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Unplugged-first: a class with no computer lab can still complete the unit on paper, and a class with one shared device can extend it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written algorithm, trace table or worked conversion Pair-debug observed against the class chart A short written response to the Liberian scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		to do. 10. digital divide the gap between people who have devices, connection and skills and people who do not. 11. ethics deciding what is right and fair in the use of technology, not only what is possible. 12. project cycle define the problem, plan a solution, build it, test it, present it.				

SEMESTER ONE**GRADE: 10 PERIOD: I****TOPIC: History and Evolution of Computing**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary precisely in speech, writing and examination answers. Solve the unit's problems on paper conversions, truth tables, traces, queries, formulas or designs showing every step of working. Evaluate a technology choice for its safety, legality, cost and effect on people in Liberia.	Upon completion of this topic, learners will: 1. Define the key terms in History and Evolution of Computing precisely and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm, program or procedure connected with this unit, on paper. 3. Classify examples into the unit's two groups and justify each placement. 4. Solve a worked example step by step, showing every line of working, and state the answer in a full sentence with its unit or type. 5. Apply the unit's ideas to a fictional Liberian business, ministry, clinic or school scenario. 6. Find and correct a bug in a set of steps, a circuit, a query or a program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared test log. 8. Explain one Liberian example of this unit's idea (mobile money, the ACE submarine cable, a hospital record, a bank, the national examination system or a radio station).	Computing developed from counting aids (abacus, Napier's bones, Pascal's calculator, Leibniz's stepped reckoner) through Babbage's mechanical designs and Lovelace's first algorithm, Hollerith's punched cards, Turing's theory of computation and the electronic first generation (ENIAC), to the stored-program computer, transistors, integrated circuits, the microprocessor, personal computers, the internet and smartphones. Each step changed who could use a computer and for what. Key content: 1. abacus an ancient counting frame of beads on rods, the earliest calculating aid. 2. Analytical Engine Charles Babbage's 1830s design for a programmable mechanical computer, never fully built. 3. Ada Lovelace the mathematician who wrote the first published algorithm intended for a machine, for Babbage's engine. 4. punched card a stiff card with holes that store data or instructions, used from Hollerith's 1890 census machines to the 1970s. 5. Alan Turing the mathematician whose 1936 model of a universal machine defined what is computable. 6. ENIAC the 1945 electronic general-purpose computer built with about 18,000 vacuum tubes. 7. stored-program concept the design in which instructions and data are held in the same memory, so a machine can be reprogrammed without rewiring. 8. transistor the small solid-state switch (1947) that replaced the vacuum tube and began the second generation.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Worked demonstration on paper Pair programming and debugging Liberian case scenario Practical and period check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of The two-track timeline world computing and Liberia with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, squared paper and printed listings A class computing chart and test log Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Written for classes that may have no computer lab: every conversion, truth table, trace, query and program in the unit can be done on paper, and a shared device extends it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written trace, conversion, truth table or query with full working Pair-debug observed against the class test log A short written response to the Liberian case scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		9. integrated circuit many transistors on one silicon chip (late 1950s), the basis of the third generation. 10. microprocessor a complete processor on one chip (1971), the basis of personal computers and phones. 11. personal computer a microcomputer built for one user, from the late 1970s. 12. World Wide Web the system of linked pages invented by Tim Berners-Lee in 1989-91 that made the internet usable by the public.				

SEMESTER ONE**GRADE: 10 PERIOD: II****TOPIC: Number Systems Binary, Octal and Hexadecimal**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary precisely in speech, writing and examination answers. Solve the unit's problems on paper conversions, truth tables, traces, queries, formulas or designs showing every step of working. Evaluate a technology choice for its safety, legality, cost and effect on people in Liberia.	Upon completion of this topic, learners will: 1. Define the key terms in Number Systems Binary, Octal and Hexadecimal precisely and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm, program or procedure connected with this unit, on paper. 3. Classify examples into the unit's two groups and justify each placement. 4. Solve a worked example step by step, showing every line of working, and state the answer in a full sentence with its unit or type. 5. Apply the unit's ideas to a fictional Liberian business, ministry, clinic or school scenario. 6. Find and correct a bug in a set of steps, a circuit, a query or a program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared test log. 8. Explain one Liberian example of this unit's idea (mobile money, the ACE submarine cable, a hospital record, a bank, the national examination system or a radio station).	A number system has a base and uses place values that are powers of the base. Decimal is base 10, binary base 2, octal base 8 and hexadecimal base 16 (digits 0 9 and A F). Convert to decimal by multiplying each digit by its place value; convert from decimal by repeated division by the base, reading the remainders upwards. Binary groups of four bits map directly to hexadecimal digits, and groups of three to octal. Key content: 1. number system a way of writing numbers using a fixed set of digits and place values based on a base. 2. base the number of different digits a system uses and the multiplier between neighbouring place values; also called the radix. 3. decimal the base-10 system with digits 0 to 9 and place values 1, 10, 100, 1,000. 4. binary the base-2 system with digits 0 and 1 and place values 1, 2, 4, 8, 16. 5. octal the base-8 system with digits 0 to 7, in which one digit stands for three binary bits. 6. hexadecimal the base-16 system with digits 0 to 9 and A to F, in which one digit stands for four binary bits. 7. place value the worth of a digit's position, a power of the base counting from the right. 8. most significant bit the leftmost bit of a binary number, which carries the largest place value. 9. least significant bit the rightmost bit of a binary number, which carries the place value 1. 10. nibble a group of four bits, which is exactly one hexadecimal digit.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Worked demonstration on paper Pair programming and debugging Liberian case scenario Practical and period check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of Place-value cards and the division ladder with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, squared paper and printed listings A class computing chart and test log Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Written for classes that may have no computer lab: every conversion, truth table, trace, query and program in the unit can be done on paper, and a shared device extends it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written trace, conversion, truth table or query with full working Pair-debug observed against the class test log A short written response to the Liberian case scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		11. subscript the small base written after a number to show its system, as in 1011 or 2F . 12. repeated division the method of converting a decimal number to another base by dividing by the base and collecting the remainders.				

SEMESTER ONE**GRADE: 10 PERIOD: III****TOPIC: Binary Arithmetic and Data Storage Units**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary precisely in speech, writing and examination answers. Solve the unit's problems on paper conversions, truth tables, traces, queries, formulas or designs showing every step of working. Evaluate a technology choice for its safety, legality, cost and effect on people in Liberia.	Upon completion of this topic, learners will: 1. Define the key terms in Binary Arithmetic and Data Storage Units precisely and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm, program or procedure connected with this unit, on paper. 3. Classify examples into the unit's two groups and justify each placement. 4. Solve a worked example step by step, showing every line of working, and state the answer in a full sentence with its unit or type. 5. Apply the unit's ideas to a fictional Liberian business, ministry, clinic or school scenario. 6. Find and correct a bug in a set of steps, a circuit, a query or a program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared test log. 8. Explain one Liberian example of this unit's idea (mobile money, the ACE submarine cable, a hospital record, a bank, the national examination system or a radio station).	Binary addition follows $0+0=0$, $0+1=1$, $1+1=10$ (carry 1) and $1+1+1=11$. Fixed word lengths cause overflow when a result is too large. Two's complement represents negative numbers: invert the bits and add 1; the leftmost bit then acts as the sign. Subtraction is done by adding the two's complement. Storage units: 8 bits = 1 byte; 1 KB = 1,000 bytes (or 1,024 in binary units), and so on to MB, GB and TB. Key content: 1. binary addition adding binary numbers column by column from the right, carrying 1 whenever a column sums to 2 or more. 2. carry the 1 moved to the next column to the left when a column's sum is 2 (10) or 3 (11). 3. word length the fixed number of bits a computer uses for one number, such as 8, 16, 32 or 64. 4. overflow an error that occurs when a result needs more bits than the word length allows, so the leading bit is lost. 5. sign bit the leftmost bit of a signed binary number, 0 for positive and 1 for negative. 6. two's complement the standard way of storing negative numbers: invert every bit of the positive value and add 1. 7. range the smallest and largest values a word can hold; 8-bit two's complement runs from -128 to +127. 8. bit a single binary digit, 0 or 1. 9. byte eight bits; enough for one ASCII character or one number from 0 to 255. 10. kilobyte about a thousand bytes (1,000 in decimal units; 1,024 in binary units). 11. megabyte about a million bytes; a gigabyte is about a thousand	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Worked demonstration on paper Pair programming and debugging Liberian case scenario Practical and period check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of Column addition with carry cards and the complement flip with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, squared paper and printed listings A class computing chart and test log Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Written for classes that may have no computer lab: every conversion, truth table, trace, query and program in the unit can be done on paper, and a shared device extends it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written trace, conversion, truth table or query with full working Pair-debug observed against the class test log A short written response to the Liberian case scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		megabytes and a terabyte about a thousand gigabytes. 12. binary shift moving every bit left or right; a left shift by one multiplies by 2, a right shift divides by 2.				

SEMESTER TWO**GRADE: 10 PERIOD: IV****TOPIC: Logic Gates and Boolean Algebra**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary precisely in speech, writing and examination answers. Solve the unit's problems on paper conversions, truth tables, traces, queries, formulas or designs showing every step of working. Evaluate a technology choice for its safety, legality, cost and effect on people in Liberia.	Upon completion of this topic, learners will: 1. Define the key terms in Logic Gates and Boolean Algebra precisely and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm, program or procedure connected with this unit, on paper. 3. Classify examples into the unit's two groups and justify each placement. 4. Solve a worked example step by step, showing every line of working, and state the answer in a full sentence with its unit or type. 5. Apply the unit's ideas to a fictional Liberian business, ministry, clinic or school scenario. 6. Find and correct a bug in a set of steps, a circuit, a query or a program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared test log. 8. Explain one Liberian example of this unit's idea (mobile money, the ACE submarine cable, a hospital record, a bank, the national examination system or a radio station).	Logic gates take binary inputs and give a binary output. AND outputs 1 only when all inputs are 1; OR outputs 1 when any input is 1; NOT inverts; NAND and NOR are AND and OR followed by NOT; XOR outputs 1 when the inputs differ. Truth tables list every input combination (2 rows for n inputs). Boolean expressions use for AND, + for OR and a bar for NOT, and follow laws such as $A + 0 = A$, $A \cdot 1 = A$ and De Morgan's laws. Key content: 1. logic gate an electronic circuit that takes one or more binary inputs and produces one binary output according to a rule. 2. AND gate a gate whose output is 1 only when every input is 1; written $A \cdot B$ or AB. 3. OR gate a gate whose output is 1 when at least one input is 1; written $A + B$. 4. NOT gate a gate with one input whose output is the opposite of the input; written $\bar{A}$. 5. NAND gate an AND gate followed by a NOT; output 0 only when every input is 1. 6. NOR gate an OR gate followed by a NOT; output 1 only when every input is 0. 7. XOR gate exclusive OR: output 1 when the inputs are different, 0 when they are the same; written $A \oplus B$. 8. truth table a table listing every combination of inputs and the resulting output of a gate or circuit. 9. Boolean algebra the algebra of the values 0 and 1 using AND, OR and NOT, developed by George Boole. 10. Boolean expression a formula built from variables, AND (), OR (+) and NOT (bar), such as $A \cdot B$	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Worked demonstration on paper Pair programming and debugging Liberian case scenario Practical and period check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of Human logic gates and the generator-alarm circuit with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, squared paper and printed listings A class computing chart and test log Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Written for classes that may have no computer lab: every conversion, truth table, trace, query and program in the unit can be done on paper, and a shared device extends it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written trace, conversion, truth table or query with full working Pair-debug observed against the class test log A short written response to the Liberian case scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		+ C. 11. logic circuit gates joined so that the output of one feeds the input of another to perform a task. 12. De Morgan's laws the rules that $\text{NOT}(A + B) = \text{NOT } A \cdot \text{NOT } B$ and $\text{NOT}(A \cdot B) = \text{NOT } A + \text{NOT } B$.				

SEMESTER TWO**GRADE: 10 PERIOD: V****TOPIC: Computer Architecture Inside the CPU**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary precisely in speech, writing and examination answers. Solve the unit's problems on paper conversions, truth tables, traces, queries, formulas or designs showing every step of working. Evaluate a technology choice for its safety, legality, cost and effect on people in Liberia.	Upon completion of this topic, learners will: 1. Define the key terms in Computer Architecture Inside the CPU precisely and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm, program or procedure connected with this unit, on paper. 3. Classify examples into the unit's two groups and justify each placement. 4. Solve a worked example step by step, showing every line of working, and state the answer in a full sentence with its unit or type. 5. Apply the unit's ideas to a fictional Liberian business, ministry, clinic or school scenario. 6. Find and correct a bug in a set of steps, a circuit, a query or a program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared test log. 8. Explain one Liberian example of this unit's idea (mobile money, the ACE submarine cable, a hospital record, a bank, the national examination system or a radio station).	The von Neumann (stored-program) architecture has a CPU, a single main memory holding instructions and data, input/output, and three buses: address, data and control. The CPU's control unit, arithmetic logic unit and registers (program counter, memory address register, memory data register, current instruction register, accumulator) carry out the fetch decode execute cycle. Performance depends on clock speed, number of cores, cache size and word length. Key content: 1. von Neumann architecture the design in which a single memory holds both instructions and data, connected to the CPU by buses. 2. register a very small, very fast storage location inside the CPU that holds one value during processing. 3. program counter the register that holds the address of the next instruction to be fetched; PC. 4. memory address register the register that holds the address of the memory location about to be read or written; MAR. 5. memory data register the register that holds the data just read from memory or about to be written; MDR. 6. current instruction register the register that holds the instruction currently being decoded and executed; CIR. 7. accumulator the register in which the arithmetic logic unit keeps its working results; ACC. 8. address bus the set of wires that carries a memory address from the CPU to memory, one direction only. 9. data bus the set of wires that carries data between the CPU and memory in both directions.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Worked demonstration on paper Pair programming and debugging Liberian case scenario Practical and period check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of The register-and-bus walk-through with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, squared paper and printed listings A class computing chart and test log Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Written for classes that may have no computer lab: every conversion, truth table, trace, query and program in the unit can be done on paper, and a shared device extends it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written trace, conversion, truth table or query with full working Pair-debug observed against the class test log A short written response to the Liberian case scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		10. control bus the wires that carry timing and command signals such as read, write and clock. 11. cache a small amount of very fast memory inside or beside the CPU that keeps recently used instructions and data. 12. core one complete processing unit; a multi-core CPU has several that can run instructions at the same time.				

SEMESTER TWO**GRADE: 10 PERIOD: VI****TOPIC: System Software and Operating Systems**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary precisely in speech, writing and examination answers. Solve the unit's problems on paper conversions, truth tables, traces, queries, formulas or designs showing every step of working. Evaluate a technology choice for its safety, legality, cost and effect on people in Liberia.	Upon completion of this topic, learners will: 1. Define the key terms in System Software and Operating Systems precisely and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm, program or procedure connected with this unit, on paper. 3. Classify examples into the unit's two groups and justify each placement. 4. Solve a worked example step by step, showing every line of working, and state the answer in a full sentence with its unit or type. 5. Apply the unit's ideas to a fictional Liberian business, ministry, clinic or school scenario. 6. Find and correct a bug in a set of steps, a circuit, a query or a program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared test log. 8. Explain one Liberian example of this unit's idea (mobile money, the ACE submarine cable, a hospital record, a bank, the national examination system or a radio station).	System software (operating systems, utilities, drivers, translators) manages the hardware and supports application software. An operating system manages processes, memory, storage and files, devices, users and security, and provides a user interface (GUI or CLI). Types include single-user, multi-user, multitasking, real-time and embedded systems. Software is licensed as proprietary, open source or free, and using unlicensed copies is illegal. Key content: 1. system software software that manages the hardware and supports applications: operating systems, utilities, drivers and translators. 2. application software software that performs tasks for the user, such as a word processor, a browser or a records program. 3. operating system the system software that manages processes, memory, files, devices and users and provides the user interface. 4. process a program that is running, together with its memory and its place in the processor's schedule. 5. scheduling the operating system's method of sharing processor time between processes. 6. multitasking running several processes apparently at once by switching the processor rapidly between them. 7. virtual memory using part of secondary storage as if it were RAM when RAM is full, at the cost of speed. 8. device driver a small program that lets the operating system control a particular piece of hardware such as a printer. 9. user interface the way a person interacts with the computer:	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Worked demonstration on paper Pair programming and debugging Liberian case scenario Practical and period check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of The operating system as a school office role play and audit with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, squared paper and printed listings A class computing chart and test log Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Written for classes that may have no computer lab: every conversion, truth table, trace, query and program in the unit can be done on paper, and a shared device extends it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written trace, conversion, truth table or query with full working Pair-debug observed against the class test log A short written response to the Liberian case scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		a graphical interface (GUI) of windows and icons, or a command-line interface (CLI) of typed commands. 10. utility program system software that does a maintenance job, such as antivirus, backup, disk clean-up or compression. 11. open-source software software whose source code is published and may be freely used, studied, changed and shared, such as Linux. 12. software licence the legal agreement that says how a copy of software may be used, installed and shared.				

SEMESTER ONE**GRADE: 11 PERIOD: I****TOPIC: Programming in Python Data Types, Operators and Expressions**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary precisely in speech, writing and examination answers. Solve the unit's problems on paper conversions, truth tables, traces, queries, formulas or designs showing every step of working. Evaluate a technology choice for its safety, legality, cost and effect on people in Liberia.	Upon completion of this topic, learners will: 1. Define the key terms in Programming in Python Data Types, Operators and Expressions precisely and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm, program or procedure connected with this unit, on paper. 3. Classify examples into the unit's two groups and justify each placement. 4. Solve a worked example step by step, showing every line of working, and state the answer in a full sentence with its unit or type. 5. Apply the unit's ideas to a fictional Liberian business, ministry, clinic or school scenario. 6. Find and correct a bug in a set of steps, a circuit, a query or a program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared test log. 8. Explain one Liberian example of this unit's idea (mobile money, the ACE submarine cable, a hospital record, a bank, the national examination system or a radio station).	Python values have types: int, float, str and bool. Arithmetic operators include +, -, *, / (true division), // (integer division), % (remainder) and the power operator, written as two asterisks; comparison operators give booleans; and, or and not combine them. Precedence runs from the power operator through *, /, //, % to + and -, then comparisons, then not, and, or; brackets override. Input arrives as a string and is converted with int() or float(); output uses print() with formatting. Key content: 1. expression a combination of values, variables and operators that Python evaluates to one value. 2. data type the kind of a value, which decides what operations are allowed: int, float, str or bool. 3. int the integer type for whole numbers such as 42 or -7. 4. float the type for numbers with a decimal point such as 3.5 or 37.0. 5. str the string type for text in quotation marks such as "Monrovia". 6. bool the boolean type with exactly two values, True and False. 7. arithmetic operator one of +, -, *, / (division), // (integer division), % (remainder) and the power operator, written as two asterisks. 8. comparison operator one of the six symbols equal ==, not equal !=, less than, greater than, less than or equal, greater than or equal that compare two values and give a bool. 9. logical operator and, or and not, which combine or reverse boolean values. 10. operator precedence the fixed order in which Python applies operators in an expression when there are no brackets.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Worked demonstration on paper Pair programming and debugging Liberian case scenario Practical and period check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of Expression evaluation cards and the type column with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, squared paper and printed listings A class computing chart and test log Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Written for classes that may have no computer lab: every conversion, truth table, trace, query and program in the unit can be done on paper, and a shared device extends it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written trace, conversion, truth table or query with full working Pair-debug observed against the class test log A short written response to the Liberian case scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		11. type conversion changing a value from one type to another with int(), float(), str() or bool(). 12. constant a variable whose value is not meant to change, written in capitals by convention, such as VAT_RATE = 0.1.				

SEMESTER ONE**GRADE: 11 PERIOD: II****TOPIC: Python Control Structures and Functions**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary precisely in speech, writing and examination answers. Solve the unit's problems on paper conversions, truth tables, traces, queries, formulas or designs showing every step of working. Evaluate a technology choice for its safety, legality, cost and effect on people in Liberia.	Upon completion of this topic, learners will: 1. Define the key terms in Python Control Structures and Functions precisely and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm, program or procedure connected with this unit, on paper. 3. Classify examples into the unit's two groups and justify each placement. 4. Solve a worked example step by step, showing every line of working, and state the answer in a full sentence with its unit or type. 5. Apply the unit's ideas to a fictional Liberian business, ministry, clinic or school scenario. 6. Find and correct a bug in a set of steps, a circuit, a query or a program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared test log. 8. Explain one Liberian example of this unit's idea (mobile money, the ACE submarine cable, a hospital record, a bank, the national examination system or a radio station).	Python selection uses if, elif and else; iteration uses for with range() or over a sequence, and while with a condition; break leaves a loop and continue skips to the next pass. Functions are defined with def name(parameters);, return a value with return, and are called by name with arguments. Variables inside a function are local; parameters pass values in. Functions make programs modular, testable and reusable. Key content: 1. function a named block of code defined with def that performs one job and can be called from elsewhere. 2. parameter a variable named in a function's definition that receives a value when the function is called. 3. argument the actual value passed to a function when it is called. 4. return value the value a function hands back to the code that called it, using the return statement. 5. local variable a variable created inside a function that exists only while the function runs. 6. global variable a variable created outside all functions that can be read anywhere in the program. 7. range() the built-in function that produces a sequence of integers: range(stop), range(start, stop) or range(start, stop, step). 8. nested loop a loop placed inside the body of another loop, so the inner loop runs completely for each pass of the outer. 9. break the statement that leaves the current loop immediately. 10. continue the statement that skips the rest of the current pass and goes to the next iteration. 11. docstring a string on the first line of a function that describes what it does, its parameters and its return	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Worked demonstration on paper Pair programming and debugging Liberian case scenario Practical and period check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of Function cards define, call, trace with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, squared paper and printed listings A class computing chart and test log Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Written for classes that may have no computer lab: every conversion, truth table, trace, query and program in the unit can be done on paper, and a shared device extends it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written trace, conversion, truth table or query with full working Pair-debug observed against the class test log A short written response to the Liberian case scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		value. 12. modular programming building a program from small functions that each do one job, so it is easier to test, read and reuse.				

SEMESTER ONE**GRADE: 11 PERIOD: III****TOPIC: Lists, Strings and Simple Algorithms**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary precisely in speech, writing and examination answers. Solve the unit's problems on paper conversions, truth tables, traces, queries, formulas or designs showing every step of working. Evaluate a technology choice for its safety, legality, cost and effect on people in Liberia.	Upon completion of this topic, learners will: 1. Define the key terms in Lists, Strings and Simple Algorithms precisely and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm, program or procedure connected with this unit, on paper. 3. Classify examples into the unit's two groups and justify each placement. 4. Solve a worked example step by step, showing every line of working, and state the answer in a full sentence with its unit or type. 5. Apply the unit's ideas to a fictional Liberian business, ministry, clinic or school scenario. 6. Find and correct a bug in a set of steps, a circuit, a query or a program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared test log. 8. Explain one Liberian example of this unit's idea (mobile money, the ACE submarine cable, a hospital record, a bank, the national examination system or a radio station).	Lists store ordered collections indexed from 0 and support append, insert, remove, len(), slicing and iteration; strings are immutable sequences of characters with methods such as upper(), lower(), split(), strip() and find(). Standard algorithms on lists traversal, counting, finding the maximum or minimum, linear search and bubble sort are written with loops and traced by hand. Key content: 1. list an ordered collection of values stored under one name, written in square brackets, such as marks = [62, 48, 75]. 2. index the position of an item in a list or string, counted from 0. 3. slice a part of a list or string taken by index range, such as name[0:3] for the first three characters. 4. append() the list method that adds a value to the end of the list. 5. len() the built-in function that returns the number of items in a list or characters in a string. 6. traversal visiting every item of a list once, usually with a for loop. 7. string method a function attached to a string, such as upper(), lower(), strip(), split() or find(). 8. immutable cannot be changed in place; strings are immutable, so a changed string is a new string. 9. linear search checking each item in turn from the first until the target is found or the list ends. 10. bubble sort a sorting algorithm that repeatedly passes through the list swapping neighbouring items that are out of order. 11. swap exchanging the values at two positions, in Python a[j], a[j+1] = a[j+1], a[j].	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Worked demonstration on paper Pair programming and debugging Liberian case scenario Practical and period check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of Index cards, a search and a bubble sort by hand with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, squared paper and printed listings A class computing chart and test log Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Written for classes that may have no computer lab: every conversion, truth table, trace, query and program in the unit can be done on paper, and a shared device extends it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written trace, conversion, truth table or query with full working Pair-debug observed against the class test log A short written response to the Liberian case scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		12. two-dimensional list a list of lists, used for a table such as marks for several students in several subjects.				

SEMESTER TWO**GRADE: 11 PERIOD: IV****TOPIC: Relational Databases and SQL**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary precisely in speech, writing and examination answers. Solve the unit's problems on paper conversions, truth tables, traces, queries, formulas or designs showing every step of working. Evaluate a technology choice for its safety, legality, cost and effect on people in Liberia.	Upon completion of this topic, learners will: 1. Define the key terms in Relational Databases and SQL precisely and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm, program or procedure connected with this unit, on paper. 3. Classify examples into the unit's two groups and justify each placement. 4. Solve a worked example step by step, showing every line of working, and state the answer in a full sentence with its unit or type. 5. Apply the unit's ideas to a fictional Liberian business, ministry, clinic or school scenario. 6. Find and correct a bug in a set of steps, a circuit, a query or a program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared test log. 8. Explain one Liberian example of this unit's idea (mobile money, the ACE submarine cable, a hospital record, a bank, the national examination system or a radio station).	A relational database stores data in linked tables: each table has a primary key and relationships are made with foreign keys (one-to-many, many-to-many via a link table). Normalisation removes repeated data. SQL retrieves data with SELECT FROM WHERE ORDER BY, joins tables with JOIN ON, summarises with COUNT, SUM and AVG, and changes data with INSERT, UPDATE and DELETE. Queries are checked by hand on sample tables. Key content: 1. relational database a database of several tables linked by keys so that each fact is stored once. 2. primary key the field that uniquely identifies each record in its own table. 3. foreign key a field in one table that holds the primary key of a record in another table, creating a link. 4. relationship the link between two tables, usually one-to-many, such as one student to many payments. 5. normalisation designing tables so that data is not repeated and each fact is stored in one place. 6. SQL Structured Query Language, the standard language for defining, querying and changing relational databases. 7. SELECT the SQL statement that retrieves chosen columns from one or more tables. 8. WHERE the SQL clause that keeps only the rows that satisfy a condition. 9. ORDER BY the SQL clause that sorts the result by one or more columns, ascending or descending. 10. JOIN the SQL operation that combines rows from two tables where their key fields match.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Worked demonstration on paper Pair programming and debugging Liberian case scenario Practical and period check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of Two paper tables and a query desk with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, squared paper and printed listings A class computing chart and test log Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Written for classes that may have no computer lab: every conversion, truth table, trace, query and program in the unit can be done on paper, and a shared device extends it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written trace, conversion, truth table or query with full working Pair-debug observed against the class test log A short written response to the Liberian case scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		11. aggregate function an SQL function that summarises many rows into one value, such as COUNT, SUM, AVG, MAX and MIN. 12. data redundancy the same data stored in more than one place, which wastes space and leads to inconsistencies.				

SEMESTER TWO**GRADE: 11 PERIOD: V****TOPIC: Spreadsheets for Analysis Functions, Charts and Models**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary precisely in speech, writing and examination answers. Solve the unit's problems on paper conversions, truth tables, traces, queries, formulas or designs showing every step of working. Evaluate a technology choice for its safety, legality, cost and effect on people in Liberia.	Upon completion of this topic, learners will: 1. Define the key terms in Spreadsheets for Analysis Functions, Charts and Models precisely and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm, program or procedure connected with this unit, on paper. 3. Classify examples into the unit's two groups and justify each placement. 4. Solve a worked example step by step, showing every line of working, and state the answer in a full sentence with its unit or type. 5. Apply the unit's ideas to a fictional Liberian business, ministry, clinic or school scenario. 6. Find and correct a bug in a set of steps, a circuit, a query or a program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared test log. 8. Explain one Liberian example of this unit's idea (mobile money, the ACE submarine cable, a hospital record, a bank, the national examination system or a radio station).	Analytical spreadsheet work uses relative and absolute references (\$A\$1), logical functions (IF, AND, OR), conditional counting and totals (COUNTIF, SUMIF, AVERAGEIF), lookups (VLOOKUP), sorting and filtering, conditional formatting, charts chosen to fit the data (bar, line, pie), and what-if models in which inputs are separated from formulas so scenarios can be compared. Key content: 1. relative reference a cell reference such as B2 that shifts when the formula is copied to another cell. 2. absolute reference a cell reference such as \$B\$1 that stays fixed when the formula is copied. 3. IF function =IF(condition, value_if_true, value_if_false), which returns one of two values depending on a test. 4. COUNTIF the function that counts the cells in a range that meet a condition, such as =COUNTIF(C2:C41, ">=50"). 5. SUMIF the function that adds the cells in a range whose matching cells meet a condition. 6. VLOOKUP the function that finds a value in the first column of a table and returns a value from another column of the same row. 7. conditional formatting a rule that changes a cell's colour or style automatically when its value meets a condition. 8. sort arranging rows in order of one or more columns. 9. filter showing only the rows that meet a condition and hiding the rest. 10. chart type the kind of chart chosen to fit the data: bar for comparisons, line for change over	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Worked demonstration on paper Pair programming and debugging Liberian case scenario Practical and period check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of The fee model with a pinned rate and a what-if with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, squared paper and printed listings A class computing chart and test log Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Written for classes that may have no computer lab: every conversion, truth table, trace, query and program in the unit can be done on paper, and a shared device extends it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written trace, conversion, truth table or query with full working Pair-debug observed against the class test log A short written response to the Liberian case scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		time, pie for parts of a whole. 11. what-if model a spreadsheet in which inputs are kept in separate cells so changing one shows its effect on all the results. 12. data validation a rule that limits what can be typed into a cell, such as a whole number between 0 and 100.				

SEMESTER TWO**GRADE: 11 PERIOD: VI****TOPIC: Networks and Data Communication Protocols and Topologies**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary precisely in speech, writing and examination answers. Solve the unit's problems on paper conversions, truth tables, traces, queries, formulas or designs showing every step of working. Evaluate a technology choice for its safety, legality, cost and effect on people in Liberia.	Upon completion of this topic, learners will: 1. Define the key terms in Networks and Data Communication Protocols and Topologies precisely and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm, program or procedure connected with this unit, on paper. 3. Classify examples into the unit's two groups and justify each placement. 4. Solve a worked example step by step, showing every line of working, and state the answer in a full sentence with its unit or type. 5. Apply the unit's ideas to a fictional Liberian business, ministry, clinic or school scenario. 6. Find and correct a bug in a set of steps, a circuit, a query or a program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared test log. 8. Explain one Liberian example of this unit's idea (mobile money, the ACE submarine cable, a hospital record, a bank, the national examination system or a radio station).	Data communication depends on agreed protocols arranged in layers: the TCP/IP model (application, transport, internet, link) with protocols such as HTTP/HTTPS, SMTP/IMAP, FTP, DNS, TCP, UDP and IP. Network topologies (star, bus, ring, mesh) trade cost against reliability. Transmission may be serial or parallel and simplex, half-duplex or full-duplex; parity bits and checksums detect errors. Firewalls, encryption and MAC/IP addressing complete the picture. Key content: 1. protocol a set of agreed rules for how devices communicate, covering format, timing, addressing and error handling. 2. TCP/IP the family of protocols the internet runs on: IP addresses and routes packets, TCP delivers them reliably in order. 3. layer one level of a protocol stack that does one job and passes data to the layer above or below. 4. HTTP and HTTPS the protocols for fetching web pages; HTTPS adds encryption with TLS. 5. DNS the Domain Name System, which translates names such as moe.gov.lr into IP addresses. 6. SMTP the protocol that sends email between servers; IMAP or POP fetch it to a client. 7. topology the physical or logical shape in which devices in a network are connected. 8. star topology every device connected to a central switch or hub. 9. mesh topology every device connected to several others, so traffic can route around a failure. 10. full-duplex communication in both directions at the same time, like a telephone call.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Worked demonstration on paper Pair programming and debugging Liberian case scenario Practical and period check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of The layered postal service and the topology stress test with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, squared paper and printed listings A class computing chart and test log Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Written for classes that may have no computer lab: every conversion, truth table, trace, query and program in the unit can be done on paper, and a shared device extends it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written trace, conversion, truth table or query with full working Pair-debug observed against the class test log A short written response to the Liberian case scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		11. parity bit an extra bit added to a group so the number of 1s is even (or odd), used to detect a single-bit error. 12. latency the time a packet takes to travel from sender to receiver, measured in milliseconds.				

SEMESTER ONE**GRADE: 12 PERIOD: I****TOPIC: Web Technologies, Client Server and Cloud Services**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary precisely in speech, writing and examination answers. Solve the unit's problems on paper conversions, truth tables, traces, queries, formulas or designs showing every step of working. Evaluate a technology choice for its safety, legality, cost and effect on people in Liberia.	Upon completion of this topic, learners will: 1. Define the key terms in Web Technologies, Client Server and Cloud Services precisely and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm, program or procedure connected with this unit, on paper. 3. Classify examples into the unit's two groups and justify each placement. 4. Solve a worked example step by step, showing every line of working, and state the answer in a full sentence with its unit or type. 5. Apply the unit's ideas to a fictional Liberian business, ministry, clinic or school scenario. 6. Find and correct a bug in a set of steps, a circuit, a query or a program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared test log. 8. Explain one Liberian example of this unit's idea (mobile money, the ACE submarine cable, a hospital record, a bank, the national examination system or a radio station).	The web works by clients (browsers) sending HTTP requests to servers, which return responses containing HTML (structure), CSS (presentation) and JavaScript (behaviour). Static pages are files; dynamic pages are generated by server-side code, often from a database. Cloud computing delivers infrastructure, platforms and software as services over the internet, trading local control for scalability, backups and cost. Key content: 1. client server model a design in which a client program asks for a service and a server program provides it over a network. 2. HTTP request the message a browser sends to a web server naming the resource it wants, such as GET /exams.html. 3. HTTP response the message a web server returns, with a status code such as 200 OK or 404 Not Found and the content. 4. CSS Cascading Style Sheets, the language that controls a page's colours, fonts, spacing and layout. 5. JavaScript the programming language that runs in the browser to make pages interactive. 6. static web page a page stored as a fixed file and sent unchanged to every visitor. 7. dynamic web page a page generated by server-side code at the moment it is requested, often from a database. 8. server-side code that runs on the web server, such as Python or PHP that queries a database and builds the page. 9. client-side code that runs in the visitor's browser, such as JavaScript that checks a form before it is sent.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Worked demonstration on paper Pair programming and debugging Liberian case scenario Practical and period check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of The request response walk and the styled school page with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, squared paper and printed listings A class computing chart and test log Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Written for classes that may have no computer lab: every conversion, truth table, trace, query and program in the unit can be done on paper, and a shared device extends it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written trace, conversion, truth table or query with full working Pair-debug observed against the class test log A short written response to the Liberian case scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		10. cloud computing using computing resources servers, storage, software provided over the internet by a third party. 11. SaaS software as a service: a complete application used through a browser, such as web email or an online document editor. 12. responsive design CSS that adapts a page's layout to the width of the screen so it works on phones and laptops.				

SEMESTER ONE**GRADE: 12 PERIOD: II****TOPIC: Computer Security, Ethics and Law**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary precisely in speech, writing and examination answers. Solve the unit's problems on paper conversions, truth tables, traces, queries, formulas or designs showing every step of working. Evaluate a technology choice for its safety, legality, cost and effect on people in Liberia.	Upon completion of this topic, learners will: 1. Define the key terms in Computer Security, Ethics and Law precisely and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm, program or procedure connected with this unit, on paper. 3. Classify examples into the unit's two groups and justify each placement. 4. Solve a worked example step by step, showing every line of working, and state the answer in a full sentence with its unit or type. 5. Apply the unit's ideas to a fictional Liberian business, ministry, clinic or school scenario. 6. Find and correct a bug in a set of steps, a circuit, a query or a program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared test log. 8. Explain one Liberian example of this unit's idea (mobile money, the ACE submarine cable, a hospital record, a bank, the national examination system or a radio station).	Information security aims at confidentiality, integrity and availability (the CIA triad). Controls include encryption (symmetric and public-key), hashing of passwords, authentication and access control, firewalls, backups, patching and audit logs. Threats include malware, denial of service, SQL injection, phishing and insider misuse. Ethics and law cover data protection and privacy, intellectual property and licensing, computer-misuse offences, and the professional responsibilities of those who handle data. Key content: 1. confidentiality keeping data readable only by those authorised to read it. 2. integrity keeping data correct, complete and unaltered except by authorised changes. 3. availability keeping systems and data usable when they are needed. 4. encryption transforming data with a key so that it is unreadable without the matching key. 5. public-key encryption encryption with a pair of keys a public key to lock and a private key to unlock used by HTTPS and digital signatures. 6. hashing turning data into a fixed-length code that cannot be reversed, used to store passwords and check files. 7. authentication proving identity with something you know, have or are: a password, a phone code, a fingerprint. 8. access control rules that decide which users may read, change or delete which data. 9. SQL injection an attack that types SQL into a form field so that a careless program runs it against the	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Worked demonstration on paper Pair programming and debugging Liberian case scenario Practical and period check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of The security audit and the ethics tribunal with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, squared paper and printed listings A class computing chart and test log Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Written for classes that may have no computer lab: every conversion, truth table, trace, query and program in the unit can be done on paper, and a shared device extends it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written trace, conversion, truth table or query with full working Pair-debug observed against the class test log A short written response to the Liberian case scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		database. 10. denial of service an attack that floods a server with requests so that real users cannot be served. 11. data protection laws and rules that govern how personal data may be collected, stored, used and shared. 12. intellectual property legal rights over creations of the mind software, writing, music including copyright and licences.				

SEMESTER ONE**GRADE: 12 PERIOD: III****TOPIC: Systems Analysis and the Software Development Life Cycle**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary precisely in speech, writing and examination answers. Solve the unit's problems on paper conversions, truth tables, traces, queries, formulas or designs showing every step of working. Evaluate a technology choice for its safety, legality, cost and effect on people in Liberia.	Upon completion of this topic, learners will: 1. Define the key terms in Systems Analysis and the Software Development Life Cycle precisely and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm, program or procedure connected with this unit, on paper. 3. Classify examples into the unit's two groups and justify each placement. 4. Solve a worked example step by step, showing every line of working, and state the answer in a full sentence with its unit or type. 5. Apply the unit's ideas to a fictional Liberian business, ministry, clinic or school scenario. 6. Find and correct a bug in a set of steps, a circuit, a query or a program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared test log. 8. Explain one Liberian example of this unit's idea (mobile money, the ACE submarine cable, a hospital record, a bank, the national examination system or a radio station).	The systems development life cycle covers feasibility study, analysis (investigation and requirements), design (data, interface, processing), implementation (coding), testing (unit, integration, system, acceptance), deployment (parallel, direct, phased, pilot) and maintenance. Analysts gather facts by interviews, questionnaires, observation and document review and model the system with data flow diagrams. Waterfall proceeds in sequence; agile works in short iterations with user feedback. Key content: 1. software development life cycle the sequence of stages by which software is planned, built, tested, delivered and maintained; the SDLC. 2. feasibility study an early check of whether a proposed system is technically, economically, legally and operationally possible. 3. requirements analysis finding out and writing down exactly what the users need the system to do. 4. systems analyst the person who investigates the current system, gathers requirements and designs the new one. 5. fact-finding gathering information by interviews, questionnaires, observation and reading existing documents. 6. data flow diagram a diagram showing processes, data stores, external entities and the flows of data between them. 7. design deciding the data structures, interfaces, processing and hardware before coding begins. 8. black-box testing testing by inputs and expected outputs without looking at the code. 9. white-box testing testing that examines the code's paths and	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Worked demonstration on paper Pair programming and debugging Liberian case scenario Practical and period check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of Analyse a real school process and plan its system with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, squared paper and printed listings A class computing chart and test log Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Written for classes that may have no computer lab: every conversion, truth table, trace, query and program in the unit can be done on paper, and a shared device extends it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written trace, conversion, truth table or query with full working Pair-debug observed against the class test log A short written response to the Liberian case scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		conditions. 10. test plan a table of test cases with input, expected result, actual result and pass or fail, covering normal, boundary and erroneous data. 11. changeover moving from the old system to the new: direct, parallel, phased or pilot. 12. agile development building software in short iterations with working versions shown to users after each.				

SEMESTER TWO**GRADE: 12 PERIOD: IV****TOPIC: Algorithm Efficiency Searching, Sorting and Big-O**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary precisely in speech, writing and examination answers. Solve the unit's problems on paper conversions, truth tables, traces, queries, formulas or designs showing every step of working. Evaluate a technology choice for its safety, legality, cost and effect on people in Liberia.	Upon completion of this topic, learners will: 1. Define the key terms in Algorithm Efficiency Searching, Sorting and Big-O precisely and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm, program or procedure connected with this unit, on paper. 3. Classify examples into the unit's two groups and justify each placement. 4. Solve a worked example step by step, showing every line of working, and state the answer in a full sentence with its unit or type. 5. Apply the unit's ideas to a fictional Liberian business, ministry, clinic or school scenario. 6. Find and correct a bug in a set of steps, a circuit, a query or a program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared test log. 8. Explain one Liberian example of this unit's idea (mobile money, the ACE submarine cable, a hospital record, a bank, the national examination system or a radio station).	Algorithm efficiency describes how running time or memory grows with input size n, expressed with Big-O: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$. Linear search is $O(n)$; binary search on a sorted list is $O(\log n)$. Bubble and insertion sort are $O(n^2)$; merge sort is $O(n \log n)$. Choosing an algorithm weighs time, memory, simplicity and whether the data is already sorted. Key content: 1. efficiency how the time or memory an algorithm needs grows as the size of its input grows. 2. Big-O notation a way of writing the growth rate of an algorithm's steps as a function of input size n, ignoring constants. 3. $O(1)$ constant time: the same number of steps whatever the input size, such as reading one array element. 4. $O(\log n)$ logarithmic time: steps grow by one each time the input doubles, such as binary search. 5. $O(n)$ linear time: steps grow in proportion to the input, such as linear search or summing a list. 6. $O(n^2)$ quadratic time: steps grow with the square of the input, such as bubble sort or nested loops over the same list. 7. binary search searching a sorted list by comparing with the middle item and discarding the half that cannot contain the target. 8. merge sort a sorting algorithm that splits the list in halves, sorts each half and merges them; $O(n \log n)$. 9. insertion sort a sorting algorithm that takes each item in turn and inserts it into its place among the already sorted items; $O(n^2)$ but fast on nearly sorted data.	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Worked demonstration on paper Pair programming and debugging Liberian case scenario Practical and period check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of Counting comparisons linear against binary, bubble against merge with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, squared paper and printed listings A class computing chart and test log Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Written for classes that may have no computer lab: every conversion, truth table, trace, query and program in the unit can be done on paper, and a shared device extends it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written trace, conversion, truth table or query with full working Pair-debug observed against the class test log A short written response to the Liberian case scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		10. best, average and worst case the fewest, typical and most steps an algorithm can take for an input of size n. 11. space complexity how the memory an algorithm needs grows with the input size. 12. trade-off accepting a cost in one measure (memory, simplicity) to gain in another (speed).				

SEMESTER TWO**GRADE: 12 PERIOD: V****TOPIC: Artificial Intelligence, Data Science and Emerging Technologies**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary precisely in speech, writing and examination answers. Solve the unit's problems on paper conversions, truth tables, traces, queries, formulas or designs showing every step of working. Evaluate a technology choice for its safety, legality, cost and effect on people in Liberia.	Upon completion of this topic, learners will: 1. Define the key terms in Artificial Intelligence, Data Science and Emerging Technologies precisely and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm, program or procedure connected with this unit, on paper. 3. Classify examples into the unit's two groups and justify each placement. 4. Solve a worked example step by step, showing every line of working, and state the answer in a full sentence with its unit or type. 5. Apply the unit's ideas to a fictional Liberian business, ministry, clinic or school scenario. 6. Find and correct a bug in a set of steps, a circuit, a query or a program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared test log. 8. Explain one Liberian example of this unit's idea (mobile money, the ACE submarine cable, a hospital record, a bank, the national examination system or a radio station).	Machine learning trains a model on data to make predictions; supervised learning uses labelled examples, unsupervised learning finds structure without labels. A model is evaluated on separate test data by accuracy and other measures; bias in training data produces biased predictions. Data science covers collecting, cleaning, exploring and visualising data. Emerging technologies IoT, blockchain, robotics, generative AI bring benefits, risks and questions of ethics, accountability and access. Key content: 1. machine learning building a model that improves at a task by learning from data rather than by following hand-written rules. 2. model the learned function that turns inputs, such as a leaf photo or a patient's symptoms, into a prediction. 3. training data the labelled examples a model learns from. 4. test data examples kept aside during training and used afterwards to measure how well the model performs on unseen cases. 5. supervised learning learning from examples that carry the right answer, such as photos labelled 'diseased' or 'healthy'. 6. unsupervised learning finding groups or patterns in data that has no labels, such as clustering clinics by their case profiles. 7. classification predicting a category, such as pass or fail, fever or no fever. 8. regression predicting a number, such as next month's rainfall or a student's mark. 9. accuracy the share of predictions that are correct; one measure among several, misleading	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Worked demonstration on paper Pair programming and debugging Liberian case scenario Practical and period check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of Train, test and audit a paper classifier with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, squared paper and printed listings A class computing chart and test log Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Written for classes that may have no computer lab: every conversion, truth table, trace, query and program in the unit can be done on paper, and a shared device extends it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written trace, conversion, truth table or query with full working Pair-debug observed against the class test log A short written response to the Liberian case scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		when classes are unbalanced. 10. bias systematic error in a model caused by unrepresentative or prejudiced training data. 11. big data data too large, fast or varied for ordinary tools, such as a mobile network's call records. 12. generative AI models that produce new text, images or code from a prompt, trained on very large collections of existing work.				

SEMESTER TWO**GRADE: 12 PERIOD: VI****TOPIC: The Computing Project and Examination Preparation**

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
Learners are able to: Use the unit's computing vocabulary precisely in speech, writing and examination answers. Solve the unit's problems on paper conversions, truth tables, traces, queries, formulas or designs showing every step of working. Evaluate a technology choice for its safety, legality, cost and effect on people in Liberia.	Upon completion of this topic, learners will: 1. Define the key terms in The Computing Project and Examination Preparation precisely and use each in a sentence of your own. 2. Write and dry-run (trace) an algorithm, program or procedure connected with this unit, on paper. 3. Classify examples into the unit's two groups and justify each placement. 4. Solve a worked example step by step, showing every line of working, and state the answer in a full sentence with its unit or type. 5. Apply the unit's ideas to a fictional Liberian business, ministry, clinic or school scenario. 6. Find and correct a bug in a set of steps, a circuit, a query or a program without blaming the person who wrote it. 7. Work in a pair as driver and navigator, swapping roles after five minutes, and keep a shared test log. 8. Explain one Liberian example of this unit's idea (mobile money, the ACE submarine cable, a hospital record, a bank, the national examination system or a radio station).	The computing project follows the life cycle end to end definition, requirements, design, implementation, testing, evaluation and report with evidence at each stage. Examination preparation covers the paper's structure, command words (state, describe, explain, compare, evaluate), showing working for conversions and traces, definitions with examples, timing by marks, and a revision timetable across the whole syllabus. Key content: 1. problem definition a short statement of the problem, who has it, and what a solution must achieve. 2. requirements numbered statements of what the system shall do and how well, against which it will be tested and evaluated. 3. design documentation the diagrams, table designs, screen sketches and algorithms produced before coding. 4. test plan the table of test cases normal, boundary, erroneous with expected and actual results and evidence. 5. evaluation an honest judgement of how far the finished system meets each requirement, with limitations and improvements. 6. technical documentation the description of the code, data and setup that lets another developer maintain the system. 7. user documentation the guide that lets a user operate the system without the developer present. 8. command word the verb in an examination question that tells you what kind of answer is wanted: state, describe, explain, compare, evaluate. 9. mark allocation the number of marks for a question, which shows how many points and how much time	Inclusive and differentiated Learning: Individual seat work or group work in mixed groups, according to abilities, gender and learning styles: Lesson focus: Worked demonstration on paper Pair programming and debugging Liberian case scenario Practical and period check 1. Computing circle: greet, take attendance, and say the two rules. 2. Paper-first demonstration of The project sprint and the marked mock with a trace on the board. 3. Pair programming: driver and navigator swap after five minutes. 4. Debug a broken set of steps together and record the bug on the class chart. 5. Sort the unit's examples into two groups and defend one choice. 6. Act the Liberian scenario and agree a safe, fair and lawful response.	Materials: Chalkboard or reusable paper Word cards for the key terms Trace-table sheets, squared paper and printed listings A class computing chart and test log Pencils, rulers and scrap paper Optional: one shared school computer or the teacher's device, kept dry and in sight Source status: Original teaching resource; not an official curriculum transcription. Written for classes that may have no computer lab: every conversion, truth table, trace, query and program in the unit can be done on paper, and a shared device extends it. Digital research links: None listed by design the course runs fully offline.	Expected competencies: Effective Communication Algorithmic and Computational Thinking Digital Skills Research and Problem Solving Collaborative Debugging	Assessment strategies: Tools that can be used to check the competencies: Oral definition of three key terms A written trace, conversion, truth table or query with full working Pair-debug observed against the class test log A short written response to the Liberian case scenario

OUTCOMES	OBJECTIVES	CONTENTS	ACTIVITIES	MATERIALS / RESOURCES	COMPETENCIES	ASSESSMENT
		it deserves. 10. trace table the step-by-step record of variable values used to answer algorithm questions and to test programs. 11. revision timetable a plan spreading the whole syllabus over the weeks available, with past-paper practice. 12. past paper an earlier examination paper used to practise timing, command words and the examiner's style.				